

The Machine, Part 1: Paisa ani Machine

August 27, 2026

THE MACHINE

PART 1 OF 5: पैसा आणि MACHINE

हे पुस्तक पाच छोट्या पुस्तकांमध्ये आहे. हे पहिलं.

पाच PARTS चा नकाशा

[तुम्ही इथे आहात]

PART 1	पैसा आणि MACHINE	पैसा का आहे, computer म्हणजे काय, सगळं numbers कसं आहे
PART 2	MACHINE शी बोलणं	languages, operating system, software बनतं कसं
PART 3	MACHINES चं जाळं	network, address, internet
PART 4	DATA आणि आठवणी	storage, database, शोधणं
PART 5	जगाला SERVE करणं	scale, cloud, खर्च, पूर्ण जोड

हे पुस्तक कशासाठी आहे

तुम्हाला पैसे कमवायचे आहेत. तुम्ही जगाला चार levels मध्ये बघितलं आहे:

LEVEL 1	माणसाची गरज	भूक, आरोग्य, ज्ञान, सुरक्षा
LEVEL 2	BUSINESS	जो गरज पूर्ण करून पैसे घेतो
LEVEL 3	TECHNOLOGY	जी business ला ताकद देते
LEVEL 4	AI	जी technology ला ताकद देते

तुम्हाला Level 4 वर काम करायचं आहे, म्हणजे खालचे सगळे levels serve करता येतील. पण Level 4 हवेत तरंगत नाही. तो Level 3 वर उभा आहे. म्हणून आधी machine समजून घ्यावी लागेल. हे पाच parts तेच करतात.

वाचून झाल्यावर तुम्ही चार गोष्टी करू शकाल:

1. कुठल्याही business problem साठी tech solution चं design स्वतः विचार करणं.
2. AI tools कडून काम काढून घेणं, जसं एक engineer काढतो.
3. कुठलाही business बघून सांगणं की tech ने पैसा कुठे बनेल.
4. Engineers आणि tech बातम्यांची भाषा पूर्ण समजणं. कोणी तुम्हाला मूर्ख बनवू शकणार नाही.

हे पुस्तक कसं लिहिलं आहे

सरळ बोलणं, तुमच्याशी. Lecture नाही. प्रत्येक गोष्टीसाठी आधी problem, मग समज, मग नाव. नाव नेहमी शेवटी येतं, कारण नाव आधी आलं की डोकं विचार करणं बंद करतं.

आणि प्रत्येक मोठ्या कल्पनेला रोजच्या जगातली एक उपमा आहे: truck आणि रस्ते, ढाबा आणि recipe, गाव आणि बाजार, post office आणि bank. Machine नवीन आहे; तिच्या आतल्या कल्पना हजारो वर्ष जुन्या आहेत.

प्रत्येक chapter चा ढाचा

CHAPTER	आधी problem, मग समज; नावं आणि खरी उदाहरणं मजकुरातच, ठळक अक्षरात
इथे लोक काय चुकीचं समजतात	जिथे लोक नेहमी घसरतात
MAP वर स्वतः बघा	ही गोष्ट Level 1-4 मध्ये कुठे, पैसा कुठे
विचार करा	5 मिनिटांचं काम, तुमच्या phone/laptop वर एक प्रश्न, उत्तर लगेच खाली

प्रत्येक chapter च्या नावासोबत एक खूण आहे. **[SPINE]** म्हणजे कणा, वगळू नका. **[DEPTH]** म्हणजे खोली, पहिल्या वेळेस वगळू शकता, नंतर परत या.

एकच सूचना

घाई करू नका. दिवसाला दोन-तीन chapters पुरे. प्रश्नाचं उत्तर आधी स्वतः विचार करा, मग खाली वाचा. जी गोष्ट समजली नाही, तिथे थांबा, पुढचं सगळं तिच्यावरच उभं असेल.

SECTION 0: पैसा, VALUE आणि SOCIETY

पाच chapters. हा section machine बदल नाही. ज्या गोष्टीसाठी तुम्ही machine शिकताय, त्या गोष्टीबद्दल आहे: पैसा.

हे तुमच्या business decision मध्ये कुठे येईल: जेव्हा तुम्ही कुठलीही कल्पना उचलाल, पहिला प्रश्न असेल “पैसा कुठून येणार.” याचं उत्तर नेहमी Level 1 पासून सुरू होतं: कोणाची गरज, किती मोठी, किती लोक. हा section तो प्रश्न विचारायला शिकवतो. याच्याशिवाय बाकीचे चार parts फक्त ज्ञान आहेत, कमाई नाही.

Chapter 0.1 [SPINE]: गावः पैसा exist का करतो

शंभर लोकांचं एक गाव विचार करा. पैसा नावाची गोष्ट अजून नाही. शेतकऱ्याकडे धान्य आहे, मिस्त्रीकडे हुनर, वैद्याकडे इलाज, चांभाराकडे जोडे.

पहिली व्यवस्था सरळ आहे: वस्तूच्या बदल्यात वस्तू. शेतकरी धान्य देईल, मिस्त्री छप्पर बांधेल. चाललं.

आता एक प्रसंग बघा. वैद्याने चांभाराचा हात बरा केला. तीन महिन्यांनी वैद्याला जोडे हवेत. पण यावेळेस चांभार एकदम ठणठणीत आहे, त्याला वैद्याकडून काहीच नको. मग वैद्य काय करणार? वाट बघणार की चांभार आजारी पडेल?

इथे व्यवस्था तुटली. सरळ देवाण-घेवाण तेव्हाच चालते जेव्हा दोन माणसांना एकाच वेळेस एकमेकांचं काम हवं असतं. या पहिल्या तुटीचं नाव **economists** ठेवतात **double coincidence of wants**: दोघांच्या गरजा एकाच क्षणी जुळल्या पाहिजेत. शंभर लोकांच्या गावातही हे क्वचित जुळतं. ही तूट लक्षात ठेवा.

दुसरी व्यवस्था स्वतः काढा. थांबा, खरंच विचार करा. काय कराल?

बरेच लोक म्हणतात: लिहून ठेवा. एक वही बनवा. वैद्याने चांभाराचं काम केलं, लिहून ठेवा, नंतर वसूल करेल. चांगलं उत्तर आहे, जगाने हे केलंही आहे; अशा वहीचं नाव **ledger**. पण दोन नवे प्रश्न उभे राहतात.

पहिला: वही ठेवणार कोण? कोणी एक माणूस. आणि ज्या दिवशी त्याने स्वतःच्या नावापुढे थोडं जास्त लिहिलं? दुकानाच्या गल्ल्यावर बसलेला माणूसच हिशोब लिहितो, तेव्हा हिशोबावर विश्वास म्हणजे माणसावर विश्वास. दुसरा: शंभर लोकांत एक वही चालली. पण दहा हजार लोक, वेगवेगळी गावं, येणारे-जाणारे लोक? किती वह्या? आणि कोणाचा कोणावर विश्वास?

म्हणजे वहीची कल्पना बरोबर आहे, तिचं रूप चुकीचं आहे. आता तिसरी व्यवस्था विचार करा. अशी कुठली गोष्ट असेल जी चांभार घेईल, त्याला इलाज अजिबात नको असला तरी? तो ती का घेईल? फक्त एवढ्यासाठी, की त्याला खात्री आहे की बाकी सगळेही ती घेतील. ती गोष्ट त्याच्या स्वतःच्या कुठल्याच कामाची नाही. तिची पूर्ण किंमत एकाच गोष्टीत आहे: पुढे चालवता येईल हा विश्वास.

आता स्वतः काढा, त्या गोष्टीत काय-काय हवं:

1. सडू नये, नाहीतर ठेवता ठेवता संपेल.
2. उचलायला सोपी हवी, जड दगडाने काम चालणार नाही.
3. नकली बनवणं अवघड हवं, नाहीतर कोणीही न देता सगळं घेईल.

4. तुकड्यांत वाटता यावी, छोटी गोष्टही विकत घ्यावी लागते.
5. सगळ्यांनी ती मानली पाहिजे. एकाने जरी नाकारली तरी साखळी तुटते.

तुम्ही आता **पैसा** invent केला. पैसा म्हणजे चालती-फिरती वही: असा ledger ज्यात विश्वास कोणा एका माणसाच्या हातात नाही, सगळ्यांमध्ये वाटलेला आहे. तुमचा **UPI** बघा: 50 रुपये पाठवता तेव्हा हातातून हातात काहीच जात नाही; एका वहीत एवढंच लिहिलं जातं की तुमच्या नावापुढे 50 कमी, त्याच्या नावापुढे 50 जास्त. ती वही banks आणि NPCI ठेवतात. जग वहीच्या कल्पनेवर परत आलं; फरक एवढाच की वही ठेवणारा आता एक माणूस नाही, एक system आहे जिला सगळे मानतात.

इथे लोक काय चुकीचं समजतात

लोकांना वाटतं पैसा म्हणजे value आहे. नाही. पैसा value ची नोंद आहे. नोट स्वतः कागद आहे; तिची किंमत या विश्वासात आहे की पुढचा माणूस ती घेईल. Post office चं stamp आठवा: तो छोटा कागद पत्र पोहोचवत नाही, तो फक्त “फी भरली आहे” ची नोंद आहे; system मेली तर stamp रद्दी. तसंच ज्या देशांमध्ये लोकांचा विश्वास तुटला, तिथे लोक पोत्यांमध्ये नोटा भरून भाकरी विकत घ्यायला गेले. कागद कागदच राहिला.

MAP वर

हा पूर्ण chapter Level 1 वर उभा आहे: माणसाची गरज आणि त्यांच्यामधली देवाण-घेवाण. पैसा इथेच जन्मतो, आणि लक्षात ठेवा, पैसा **फक्त इथेच** घुसतो, जेव्हा एक माणूस आपल्या गरजेसाठी देतो. Level 2, 3, 4 फक्त ही देवाण-घेवाण मोठी, वेगवान आणि स्वस्त करतात. जो कोणी कमवू इच्छितो, त्याची साखळी कुठेतरी Level 1 च्या एका माणसावर संपते.

स्वतः बघा (5 मिनिटं)

तुमचं UPI app उघडा. शेवटचे 5 व्यवहार बघा. प्रत्येकासाठी विचारा: कुठल्या गरजेसाठी पैसा गेला? खाणं, प्रवास, ज्ञान, अजून काही? प्रत्येक व्यवहार कुठल्यातरी गरजेवर संपेल. एकही असा मिळणार नाही जो कुठल्याही माणसाच्या गरजेवर संपत नाही.

विचार करा

1. (derivation) जर जगात फक्त एकाच वस्तूची देवाण-घेवाण असती, समजा फक्त धान्य, तर पैसा तरीही बनला असता का?

उत्तर: नाही, किंवा खूप उशिरा. पैसा त्या problem चा उपाय आहे जिथे खूप वेगवेगळ्या वस्तूंची देवाण-घेवाण आहे आणि वेळा जुळत नाहीत. एकच वस्तू असेल तर सरळ देवाण-घेवाण पुरते, double coincidence ची तूट होतच नाही. यातून एक खोल गोष्ट दिसते: पैसा तेव्हा गरजेचा होतो जेव्हा समाजात कामं वाटली जातात. जेवढी जास्त कामांची वाटणी, तेवढा गरजेचा पैसा. याच गोष्टीवर पुढचा chapter उभा आहे.

Chapter 0.2 [SPINE]: तास सारखे, कमाई वेगळी

दोन माणसं, एकच गाव, एकच वय, दोघेही मेहनती.

रवी मिस्त्री आहे. रोज 12 तास काम करतो. प्रत्येक घर त्याच्या हातांनी बनतं. ज्या दिवशी तो काम करत नाही, त्या दिवशी कमाई शून्य. वीस वर्षांनंतरही त्याचा दिवस 12 तासांचाच आहे.

अनिलने गावासाठी पाण्याच्या नळाचं design बनवलं. सहा महिने लागले. त्यानंतर नळ-योजना बांधली गेली आणि चालू झाली. आता पाणी रोज प्रत्येक घरी पोहोचतं, अनिलने हात न लावता. गाव अनिलला दर महिन्याला हिस्सा देतं. अनिल आता दुसऱ्या गावासाठी design बनवतोय.

रवी जास्त तास लावतो. अनिल जास्त कमावतो. का?

डोकं पहिलं उत्तर देतं: अनिल शिकलेला आहे. पण हे उत्तर काहीच सांगत नाही. वैद्यही शिकलेला आहे आणि रवीएवढंच कमावतो. खरं उत्तर तीन भागांत आहे. तिन्ही स्वतः काढा:

Problem किती मोठी आहे? रवी एका घराची problem सोडवतो, एका कुटुंबासाठी. अनिलने पूर्ण गावाची पाण्याची problem सोडवली, शंभर कुटुंबांसाठी. जेवढे जास्त लोक, जेवढी खोल गरज, तेवढा मोठा पैसा त्या problem च्या मागे उभा असतो. याचं नाव **market size**.

हे किती लोकांना जमतं? भिंत बांधणं गावातले वीस जण जाणतात. नळाचं design एकच जाणत होता. जी गोष्ट कमी लोकांना जमते, तिचा भाव वर जातो. बाजाराचा सरळ नियम: टंचाई. याचं नाव **scarcity**. Auto-rickshaw सगळ्यांना चालवता येते, म्हणून भाडं ठरलेलं; crane फार कमी लोकांना, म्हणून crane चाल्याचा दर तोच सांगतो.

काम एकदा, फायदा किती वेळा? हा सगळ्यात लपलेला भाग आहे. रवीची बांधणी त्या दिवशी संपते. अनिलचं design एकदा बनलं आणि आता रोज काम करतंय, अनिलशिवाय. रवी आपला वेळ विकतोय. अनिलने अशी गोष्ट बनवली जी तो झोपल्यावरही चालते. याचं नाव **leverage**.

मग सूत्र हे निघालं:

$$\text{कमाई} = \text{problem चा SIZE} \times \text{SCARCITY} \times \text{LEVERAGE}$$

या सूत्रात नीट बघा: **तास कुठेच नाहीत**. म्हणूनच 2 तासवाला billionaire असू शकतो आणि 12 तासवाला तंगीत. मेहनत चुकीच्या गोष्टीवर लागली असेल तर मेहनतीचा भाव मिळत नाही.

खऱ्या जगात हे असं दिसतं: एक surgeon घ्या आणि एक app बनवणारा. Surgeon ची skill जास्त दुर्मिळ, शिक्षण जास्त लांब. तरी जगातले सगळ्यात श्रीमंत लोक software वाले आहेत, surgeons नाहीत. का? Surgeon चं प्रत्येक operation त्याच्या स्वतःच्या हातांनी होतं, दिवसाला दोन-तीन. Leverage चा काटा शून्यावर अडकलेला. App एकदा बनते आणि 10 million (1 कोटी) लोक एकाच वेळेस वापरतात. WhatsApp विकली गेली तेव्हा (19 billion dollars) तिच्याकडे फक्त 55 कर्मचारी होते, आणि users 400 million. हे leverage आहे.

इथे लोक काय चुकीचं समजतात

लोकांना वाटतं जास्त कमवायचा रस्ता जास्त मेहनत आहे. सूत्र म्हणतं: मेहनतीचा गुणाकार तेव्हाच चालतो जेव्हा बाकीचे तीन काटे वर असतात. Rickshaw चालवणारा दुप्पट तास चालवून दुप्पट थकतो, दुप्पट श्रीमंत होत नाही; पण दहा rickshaw भाड्याने देणारा झोपेतही कमावतो. दुसरी चूक: लोक scarcity ला degree समजतात. Scarcity म्हणजे certificate नाही; त्या वेळेस, त्या जागी, ते काम किती लोकांना जमतं, एवढंच.

MAP वर

या सूत्राचा तिसरा काटा, leverage, हेच खरं कारण आहे की Level 3 (technology) एवढा पैसा बनवतो. Machine हीच ती गोष्ट आहे जी एकदा बनते आणि पुन्हा पुन्हा चालते, न थकता. म्हणूनच तुम्ही हे पुस्तक वाचताय: Level 3 आणि 4 वर leverage चा काटा सगळ्यात वर फिरतो.

स्वतः बघा (5 मिनिटं)

जगातल्या 10 सगळ्यात श्रीमंत लोकांची यादी उघडा (search: richest people). प्रत्येक नावापुढे विचारा: हा माणूस तास विकून इथे पोहोचला का? एकही मिळणार नाही. प्रत्येकाकडे अशी गोष्ट आहे जी त्याच्याशिवाय चालते: company, software, brand, factory.

विचार करा

1. (derivation) एक शिक्षक 30 मुलांना शिकवतो आणि महिन्याला X कमावतो. त्याच ज्ञानातून तो करोडो कसा कमवू शकतो? सूत्राचे तिन्ही काटे फिरवून सांगा.

उत्तर: Size: 30 मुलांऐवजी देशभरची मुलं, online. Scarcity: सगळे विषय नाहीत; ती एक गोष्ट जी फक्त तोच सगळ्यात चांगली समजावतो. Leverage: रोज class घेण्याऐवजी एकदा recorded course किंवा एक पुस्तक, जे एकदा बनतं आणि लाखो वेळा विकलं जातं. Physics Wallah हीच गोष्ट आहे: एक शिक्षक, YouTube वर recorded classes, आज billion- dollar company. ज्ञान तेच होतं; काटे फिरले.

Chapter 0.3 [SPINE]: समाजाचे चार levels

Chapter 0.1 च्या गावात परत चला, पण आता पूर्ण जग विचार करा. Billions लोक. या गर्दीत पैसा कुठून सुरू होतो आणि कुठे जातो? एक नकाशा बनवू, खालून वर. इमारतीसारखा विचार करा: प्रत्येक मजला खालच्या मजल्यावर उभा असतो, आणि पाया नेहमी जमिनीत असतो.

Level 1: माणसाची गरज. भूक, तहान, छप्पर, आरोग्य, ज्ञान, सुरक्षा, प्रवास, जोडलेपण, करमणूक, इज्जत. ही यादी हजारो वर्षांपासून जवळजवळ तीच आहे. हीच पूर्ण economy ची जमीन आहे. जगात फिरणारा प्रत्येक रुपया शेवटी कोणाच्यातरी कुठल्यातरी गरजेतून निघाला आहे.

Level 2: Business. एका माणसाची गरज एकदा पूर्ण करणं म्हणजे काम. लाखो लोकांची गरज पुन्हा पुन्हा पूर्ण करणं व्यवस्था मागतं: माणसं, सामान, हिशोब, विश्वास. त्या व्यवस्थेचं नाव business. एक बाई घरात स्वयंपाक करते ते काम; तीच बाई रोज शंभर डबे पोहोचवते ती व्यवस्था, म्हणजे business. Restaurant भुकेवर उभं आहे, hospital आरोग्यावर, school ज्ञानावर. Business म्हणजे गरजेला पैशात बदलणारी व्यवस्था.

Level 3: Technology. Business ला नेहमी तीन गोष्टींची भूक असते: जास्त लोक serve करा, कमी खर्चात, कमी चुकांसह. जी गोष्ट हे करते, ती technology. Tractor शेतकऱ्यासाठी, fridge restaurant साठी, software सगळ्यांसाठी. ढाब्याचा तंदूर पण technology आहे: एक cook च्या जागी एका वेळेस वीस रोट्या. Technology म्हणजे business ची ताकद वाढवणारा मजला.

Level 4: AI. आता technology स्वतः एक गरजवाली गोष्ट झाली आहे: तिच्यासाठी विचार करणं, लिहिणं, ठरवणं, हे आत्तापर्यंत माणसं करत होती. जी machine ही कामं करू लागली आहे, ती technology ची technology. AI Level 3 ला तेच देतो जे Level 3 ने Level 2 ला दिलं: जास्त, स्वस्त, कमी चुका.

आता सगळ्यात महत्वाचा बाण काढा:

LEVEL 4	AI	^	
LEVEL 3	TECHNOLOGY		पैसा वरच्या
LEVEL 2	BUSINESS		दिशेने वाहतो
LEVEL 1	माणसाची गरज		

पैसा **फक्त Level 1 वर** घुसतो, जेव्हा एक माणूस आपल्या गरजेसाठी देतो. Level 2 त्यातला हिस्सा घेतो. Level 3 ला Level 2 देतो. Level 4 ला Level 3 देतो. वरच्या मजल्यांचा प्रत्येक रुपया खालून चढत आलेला आहे. अशा एकावर-एक रचनेला **engineers stack** म्हणतात; पुढे “tech stack” ऐकाल तेव्हा हाच अर्थ: कोण कोणावर उभा आहे, कोण कोणाला पैसे देतो.

हे खऱ्या जगात असं दिसतं: **Zomato** घ्या. Level 1: भूक. Level 2: restaurant, आणि स्वतः Zomato चा business. Level 3: app, servers, नकाशा, payment system. Level 4: कोणतं खाणं कोणाला दाखवायचं, delivery चा रस्ता, मागणीचा अंदाज, हे AI करतो. तुम्ही 300 रुपयांचं जेवण मागवता: restaurant हिस्सा घेतं, Zomato हिस्सा घेतं, Zomato आपल्या cloud आणि software वाल्यांना देतं, ते AI चालवणाऱ्यांना देतात. एक भूक, चार मजल्यांची कमाई.

तीन गोष्टी तपासून घ्या, कारण स्वच्छ नकाशा त्या लपवतो: प्रत्येक गरज business ने पूर्ण होत नाही (आई मुलाला जेवू घालते, कुठेही पैसा फिरत नाही). प्रत्येक business ला जास्त technology लागत नाही. आणि प्रत्येक technology ला AI चांगली करत नाही (पाण्याच्या pipe मध्ये विचार करणाऱ्या machine साठी काही कामच नाही). हा नकाशा कल दाखवतो, कायदा नाही.

इथे लोक काय चुकीचं समजतात

लोकांना वाटतं पैसा technology मध्ये आहे, म्हणून tech शिका. उलटं आहे. पैसा **गरजेत** आहे. Technology फक्त ती गरज पकडण्याचा सगळ्यात लांब हात आहे. विहीर पाण्याने भरलेली असते; दोरी-पोहरा फक्त पाणी काढण्याचं साधन आहे. दोरी कितीही उत्तम असो, विहीर कोरडी असेल तर हाती काही येत नाही. जो माणूस Level 4 ची skill घेऊन बसला आहे पण कुठल्याही Level 1 गरजेशी जोडलेला नाही, तो काही कमावत नाही.

MAP वर

हा chapter स्वतःच नकाशा आहे. फोटोसारखा डोक्यात ठेवा. पुढे प्रत्येक chapter च्या शेवटी “MAP वर” याच नकाशावर बोट ठेवेल: ही गोष्ट कुठल्या level वर आहे, आणि पैसा हिच्यापर्यंत कसा पोहोचतो.

स्वतः बघा (5 मिनिटं)

Phone ची home screen उघडा. कुठलीही 3 apps निवडा. प्रत्येकासाठी लिहा: कुठली Level 1 गरज? कुठला Level 2 business? WhatsApp: जोडलेपणाची गरज, Meta चा business. असं app शोधा जे कुठल्याच गरजेवर उभं नाही. मिळणार नाही.

विचार करा

1. (derivation) AI companies (Level 4) billions कमावतायत. पण पैसा फक्त Level 1 वर घुसतो. मग OpenAI चा पैसा कुठल्या रस्त्याने, कुठल्या levels मधून येतो? साखळी लिहा.

उत्तर: एक साखळी अशी असू शकते: तुम्ही भुकेने जेवण order करता (L1 पासून पैसा घुसला). Zomato कमावतं (L2). Zomato आपल्या software आणि cloud वर खर्च करतं (L3). ते software बनवणारी team ChatGPT चं subscription घेते, code वेगाने लिहिण्यासाठी (L4). AI चा प्रत्येक रुपया अशा कुठल्यातरी साखळीने खालच्या levels मधून आलेला असतो. याचं उलटंही खरं आहे: जर एखादं AI product कुठल्याच साखळीने Level 1 पर्यंत पोहोचत नसेल, तर त्याच्याकडे येणारा पैसाच नाही.

Chapter 0.4 [SPINE]: Technology चा मजला दिसत का नाही

आज रात्री तुम्ही phone वरून जेवण order करता. तुम्हाला काय दिसतं? एक button: “Place Order.” बस.

आता मोजू, त्या एका button च्या मागे काय चालतं. तुमचा phone एक message बनवून हवेत पाठवतो. तो message शहरापलीकडच्या कुठल्यातरी building मधल्या machine पर्यंत जातो. ती machine तुमचं नाव, पत्ता, order, पैशाचा हिशोब तपासते. Bank च्या machine शी बोलून 300 रुपये इकडून तिकडे करते. Restaurant च्या screen वर order चमकतो. अजून एक machine विचार करते की कुठला delivery वाला कुठे आहे आणि कोणाला पाठवणं स्वस्त पडेल. तुमच्या नकाशावर तो चालताना दिसतो.

हे सगळं 5 seconds मध्ये झालं. आणि तुम्हाला यातलं **काहीही** दिसलं नाही.

ही चूक नाही. हे design आहे. का? दोन कारणं, दोन्ही पैशाची:

पहिलं: तुम्ही भुकेले आहात. तुम्ही जेवणाचे पैसे देताय, machine बघण्याचे नाही. ढाब्यावर जाता तेव्हा तुम्हाला गरम रोटी हवी असते; मालक तुम्हाला kitchen मध्ये नेऊन तंदूर, पीठाची गोणी आणि gas चं bill दाखवू लागला तर तुम्ही उठून दुसऱ्या ढाब्यावर जाल. जी गोष्ट ग्राहकाला दिसते आणि अडवते, ती पैसा घालवते. म्हणून जी technology जेवढी चांगली, तेवढी गायब.

दुसरं: जी गोष्ट गायब आहे, ती कोणी शिकू शकत नाही. जी शिकता येत नाही, ती बनवता येत नाही. मग दिसणारा भाग (button) स्वस्त झाला, सगळे बनवतात. न दिसणारा भाग (मागची व्यवस्था) महाग राहिला, कारण कमी लोक जाणतात. Chapter 0.2 चा scarcity चा काटा आठवा? Technology ची scarcity तिच्या न-दिसण्यातून येते.

या लपवण्याच्या कामाचं नाव engineers ठेवतात **abstraction**: आतली गुंतागुंत झाकून वर एक सोपं handle देणं. Button abstraction आहे. गाडीचं steering wheel abstraction आहे: engine मध्ये हजार भाग फिरतात, तुमच्या हातात एक चाक. UPI चा QR code abstraction आहे: banks चं पूर्ण जाळं झाकलेलं. हा शब्द या पुस्तकात शंभर वेळा येईल, कारण पूर्ण technology म्हणजे abstraction वर abstraction ची शिडी.

खऱ्या जगातलं सगळ्यात मोठं उदाहरण: **Jio** ने 2016 मध्ये फुकट data दिला आणि देश बदलला. लोकांना काय दिसलं? एक स्वस्त SIM. काय दिसलं नाही? समुद्राखाली पडलेल्या fibre च्या तारा, प्रत्येक शहरात towers, त्यांच्यामधली व्यवस्था, या सगळ्यावर लावलेले 2.5 लाख कोटी रुपये (~30 billion

dollars). Level 3 इतका मोठा होता की देशाची सगळ्यात मोठी company बनला, आणि सामान्य माणसाने तो कधी बघितलाच नाही. तुमच्या शिकण्यासाठी हीच सगळ्यात मोठी बातमी आहे: **सगळ्यात मोठ्या पैशांची कामं डोळ्यांपासून लपलेली आहेत.**

इथे लोक काय चुकीचं समजतात

लोक “technology” ऐकताच gadget विचार करतात: phone, laptop, robot. Gadget म्हणजे technology चा फक्त तो कोपरा जो विकण्यासाठी दाखवला जातो. भाजी-मंडईचं वजन-काटा दिसतो; पण मंडई चालवणारी खरी व्यवस्था म्हणजे पहाटेचे trucks, लिलाव, उधारीचे हिशोब, ते कोणाला दिसत नाहीत. खरी technology व्यवस्था आहे: numbers ठेवण्याची, पाठवण्याची, तपासण्याची, ठरवण्याची. म्हणून “tech शिकणं म्हणजे coding शिकणं” ही पण अर्धीच गोष्ट आहे. Coding एक हत्यार आहे. समजणं हे आहे की पूर्ण व्यवस्था कशी उभी राहते.

MAP वर

रुपयाचा रस्ता बघा: तुमच्या 300 रुपयांतले restaurant ला ~200, Zomato ला ~100 मिळाले. त्या 100 मधून Zomato cloud, payment gateway, नकाशा- सेवा, SMS पाठवणाऱ्यांना देतं. हे सगळे Level 3 च्या आतले business आहेत ज्यांची नावं तुम्ही कधी ऐकली नाहीत: Razorpay, AWS, Twilio. न-दिसणाऱ्या मजल्याच्या आतही पूर्ण बाजार आहेत. जिथे डोळा जात नाही, तिथे स्पर्धाही कमी असते.

स्वतः बघा (5 मिनिटं)

कुठलंही online payment करा (किंवा जुनं आठवा). कागदावर लिहा: या 5 seconds मध्ये किती वेगळ्या companies नी काम केलं असेल? मोजा: तुमचा phone बनवणारी, network वाली, app वाली, तुमची bank, payment gateway, समोरच्याची bank, NPCI. एक tap, सातपेक्षा जास्त companies. प्रत्येकीला त्या tap मधून पैसा मिळतो.

विचार करा

1. (derivation) एखादा business आपली technology लपवण्याऐवजी पूर्ण दाखवू लागला तर (प्रत्येक order वर: “आता server X शी बोलणं चालू आहे...”) त्याचं काय होईल? आणि या उत्तरातून काढा: engineer चा पगार जास्त का असतो?

उत्तर: ग्राहक पळून जातील, कारण त्यांना गरज पूर्ण करायची होती, machine समजायची नव्हती. Business दाखवण्यासाठी नाही, लपवण्यासाठी पैसे देतो. आणि जी गोष्ट सगळ्यात लपलेली आहे, ती समजणारे सगळ्यात कमी लोक आहेत. कमी लोक = scarcity = उंच पगार. Engineer ला या गोष्टीचे पैसे मिळतात की तो त्या खोलीत काम करू शकतो जिथे बाकीचं जग घुसूच शकत नाही. तुम्ही हे पुस्तक वाचून त्याच खोलीची किल्ली बनवत आहात.

Chapter 0.5 [SPINE]: मग आता आपल्याला काय उघडावं लागेल

चार chapters मध्ये तुमच्याकडे आता हे आहे:

1. पैसा म्हणजे विश्वासाची व्यवस्था, आणि **फक्त** Level 1 वर घुसतो. (0.1)
2. कमाई = size x scarcity x leverage. तास सूत्रात नाहीत. (0.2)
3. जग चार levels चा stack आहे, पैसा खालून वर वाहतो. (0.3)
4. Level 3 मुद्दाम लपवलेला आहे, आणि लपलेलं असणं हाच त्याचा भाव. (0.4)

आता एक प्रामाणिक प्रश्न: Level 1 आणि 2 तुम्हाला आधीच समजतात. गरज म्हणजे काय, business कसा चालतो, हे तुम्हाला दिसतं. जे दिसत नाही, ते Level 3 आणि 4. आणि तुमचा plan Level 4 वर काम करण्याचा आहे.

मग सरळ AI च का शिकू नये? हा प्रश्न प्रत्येकजण विचारतो, आणि इथेच सगळे घसरतात. उत्तरासाठी एक प्रश्न पुरे: AI चालतो कशावर? Computer वर. आणि computer तुम्हाला समजत नसेल, तर AI तुमच्यासाठी काय असेल? जादू. जादूच्या प्रेक्षकाला काय जमतं? बघणं, थोडं वापरणं. काय जमत नाही? Design करणं, विकणं, सुधारणं, आणि तुटल्यावर सांगणं की काय तुटलं. जादूचा प्रेक्षक बनता येतं, जादूगार नाही.

Driver आणि mechanic चा फरक हाच आहे. दोघेही गाडी चालवतात. पण गाडी रस्त्यात बंद पडली की driver फक्त उभा राहतो; mechanic bonnet उघडतो. आणि गाड्यांचा **धंदा** कोण करतं? ज्याला आतलं कळतं तो. AI च्या जगात तुम्हाला driver नाही, mechanic-मालक बऱ्याच आहे.

म्हणून रस्ता खालून आहे. Level 4 Level 3 वर उभा आहे, मग आधी Level 3: machine. हा Part 1 ते Part 5 चा प्रवास आहे:

PART 1 (हा)	machine म्हणजे काय, सगळं numbers कसं आहे
PART 2	machine ला सांगणं की काय करायचं: software
PART 3	machines चं आपसात बोलणं: internet
PART 4	लक्षात ठेवणं: data, database
PART 5	लाखो लोकांना serve करणं: scale, cloud, खर्च

यानंतर Book 2 येते: AI. तेव्हा ती जादू वाटणार नाही, machine वाटेल.

आणि एक दावा लिहून ठेवा, आपण तो Part 5 च्या शेवटी तपासू:

दावा: Machine समजल्यावर तुम्ही कुठलाही business बघून सांगू शकाल की त्यात technology ने पैसा कुठे बनेल, काय बनवणं स्वस्त आहे, काय महाग, आणि AI त्यात कुठे घुसेल.

इथे लोक काय चुकीचं समजतात

"Basics नंतर बघेन, आधी जलद रस्ता." असा रस्ता असता तर सगळे त्यावर असते. जे लोक AI tools सगळ्यात चांगले चालवतात, ते तेच आहेत ज्यांना आत काय चालतं ते माहीत आहे; म्हणून ते बरोबर गोष्ट मागू शकतात आणि चुकीचं उत्तर पकडू शकतात. Machine ची समज हीच AI ची steering आहे.

MAP वर

तुम्ही या क्षणी नकाशावर स्वतःला ठेवताय: Level 1-2 ची समज तुमच्याकडे आहे, Level 3 चा दरवाजा हे पुस्तक आहे, Level 4 पुढचं पुस्तक. ज्या दिवशी तुमच्याकडे चारही levels ची समज असेल, तुम्ही त्या छोट्या गर्दीत असाल जिला पूर्ण साखळी दिसते. साखळी बघणाराच साखळी design करू शकतो.

स्वतः बघा (5 मिनिटं)

एका कागदावर स्वतःच्या हाताने हे लिहा आणि photo काढून ठेवा: "मी हे यासाठी शिकतोय की _____." आपल्या शब्दांत भरा. Part 5 च्या शेवटच्या chapter मध्ये ही ओळ परत मागितली जाईल; तेव्हा बघा उत्तर बदललं की तेच आहे.

विचार करा

1. (derivation) वरचा तर्क उलटा करा: एका माणसाने **फक्त** Level 3 शिकला, machine चा पूर्ण expert, पण Level 1-2 (गरज, business) कधी बघितलंच नाही. त्याच्या कमाईचं काय होईल?

उत्तर: तो कमावेल, पण नेहमी दुसऱ्यासाठी. पगार मिळेल, कारण त्याची skill दुर्मिळ आहे. पण size आणि leverage चे काटे दुसरा कोणी फिरवेल: founder, ज्याला माहीत आहे कुठल्या गरजेवर काय बनवायचं. जगातले लाखो उत्तम engineers पगारावर आहेत आणि त्यांचे founders billionaires आहेत. फरक skill चा नाही, साखळीच्या पूर्ण समजेचा आहे. म्हणून या पुस्तकात प्रत्येक chapter पैशाच्या ओळीसह येतो: तुम्हाला दोन्ही डोळे हवेत, machine वालाही, पैसा वालाही.

SECTION 1: पहिला प्रश्न

सहा chapters. इथून machine उघडते: भिंतीवरच्या switch पासून या गोष्टीपर्यंत की एकच machine सगळं का करू शकते.

हे तुमच्या business decision मध्ये कुठे येईल: जेव्हा तुम्ही एखाद्या engineer कडून किंवा AI कडून product बनवून घ्याल, तेव्हा “काय शक्य आहे, काय नाही, काय स्वस्त, काय महाग” याचा अंदाज याच section च्या समजेतून येईल. जो founder machine ला जादू समजतो, तो चुकीची गोष्ट मागतो, चुकीच्या भावात, चुकीच्या माणसाकडून. जो समजतो, तो नेमकं तेच मागतो जे machine देऊ शकते.

Chapter 1.1 [SPINE]: एक switch दाबल्यावर काय होतं

भिंतीचा switch दाबा. Bulb लागला. हे पूर्ण पुस्तक याच एका घटनेच्या आत लपलं आहे, म्हणून हळूहळू उघडू.

वीज घरात तारांच्या रस्त्याने येते. तार म्हणजे एक बंद रस्ता: power station पासून निघून, bulb मधून जाऊन, परत. जोपर्यंत रस्ता पूर्ण जोडलेला आहे, वीज वाहते आणि bulb जळतो. अशा बंद रस्त्याचं नाव **circuit**. Switch त्या रस्त्यातली एक कडी आहे जी तुटते आणि जुळते. दाबला: रस्ता जुळला, वीज वाहिली, उजेड. परत दाबला: रस्ता तुटला, अंधार.

मग switch खरं काय आहे? **एका प्रश्नाचं उत्तर, जे दोनपैकी एकच असू शकतं: जुळलेला, की तुटलेला. हो, की नाही. 1, की 0.**

आता तीन उड्या मारा, प्रत्येक छोटी आहे:

पहिली: Switch बोटाने दाबला जातो. विजेने दाबता येईल का? हो, असा switch बनतो ज्याला दुसऱ्या तारेतला current दाबतो. याचा अर्थ नीट ऐका: **एक switch दुसऱ्या switch ला दाबू शकतो.** बोटाची गरज फक्त सुरुवातीला आहे.

दुसरी: एक switch दुसऱ्याला दाबू शकतो, तर हजार switch मिळून एक साखळी बनू शकते: हा दाबला तर तो दाबेल, तो दाबला तर हा उघडेला. रेल्वेचं signal-yard आठवा: एक lever ओढला की पुढे रूळ बदलतो, रूळ बदलला की पुढचा signal बदलतो; stationmaster एका जागी बसून पूर्ण yard चालवतो. साखळी design करणं म्हणजे आधीच ठरवणं की काय झाल्यावर काय होईल. हे “आधीच ठरवणं” पुढे खूप मोठं होणार आहे.

तिसरी: आजच्या switch मध्ये हलणारा भाग नाही. तो वाळूच्या धातूचा (silicon) एक तुकडा आहे जो विजेच्या इशान्यावर रस्ता उघडतो-बंद करतो. या विजेने-दाबल्या-जाणाऱ्या switch चं नाव **transistor**: माणसाचा कदाचित सगळ्यात महत्वाचा शोध. झीज नाही, आवाज नाही, आणि आकार? तुमच्या phone च्या आतल्या chip मध्ये असे **15 billion** (1,500 कोटी) transistors आहेत. नखाएवढ्या जागेत.

मग हे पुस्तक जी machine उघडतंय, ती ही आहे: **billions switches, एकमेकांना दाबत, एका design केलेल्या साखळीत.** तुमचा प्रत्येक tap, प्रत्येक message, प्रत्येक video याचाच निकाल आहे. Phone चा torch लावा: बोटाने काच दाबली, काचेखालच्या sensor ने छोटा current पाठवला, त्याने साखळी सुरू केली, साखळीच्या शेवटी एका transistor ने LED चा रस्ता जोडला. उजेड. तुम्ही भिंतीचा switch च दाबला, फक्त मध्ये billions switches होते. फरक फक्त मोजणीचा आहे, कल्पना तीच.

इथे लोक काय चुकीचं समजतात

लोकांना वाटतं computer ही कुठलीतरी विचित्र, समजण्यापलीकडची गोष्ट आहे. खरं हे आहे की तिचा प्रत्येक भाग भिंतीच्या switch इतकाच सरळ आहे. डोकं चक्रावतं ते **मोजणीने**: billions switches, billion वेळा प्रति second. पण मोजणीला घाबरणं म्हणजे समुद्राला घाबरणं कारण त्यात थेंब खूप आहेत. थेंब समजला की समुद्र समजतो.

MAP वर

Level 3 ची पूर्ण इमारत या एका भागावर उभी आहे, आणि transistor बनवणं जगातल्या सगळ्यात मोठ्या धंद्यांपैकी आहे. एक company, **TSMC** (Taiwan), जगातले सगळ्यात उत्तम transistors बनवते, आणि Apple पासून Nvidia पर्यंत सगळे तिच्या दारात रांगेत उभे आहेत. जी गोष्ट दुसऱ्या कोणालाच बनवता येत नाही = scarcity चा काटा पूर्ण वर. एक factory एकदा उभी राहते आणि billions chips काढते = leverage पण पूर्ण वर. सूत्र काम करतंय.

स्वतः बघा (5 मिनिटं)

घरचा कुठलाही switch दाबताना हा विचार करा: मी एका प्रश्नाचं उत्तर set करतोय, हो की नाही. मग phone चा torch लावा आणि स्वतःला सांगा: आत्ता मी billions switches च्या साखळीच्या टोकावर बोट ठेवलं. दोन्ही एकच गोष्ट आहेत. हे जाणवणं हेच या chapter चं खरं काम.

विचार करा

1. (derivation) “एक switch दुसऱ्या switch ला दाबू शकतो.” हे शक्य **नसतं**, बोट नेहमी लागलं असतं, तर computer चं काय झालं असतं?

उत्तर: Computer बनूच शकला नसता. प्रत्येक पावलावर माणूस लागला असता, मग machine चा वेग माणसाच्या वेगावर अडकला असता: second ला billion पावलांऐवजी second ला एक. आणि leverage शून्य झाला असता: machine ही “एकदा बनवा, चालत राहील” वाली गोष्ट राहिली नसती. पूर्ण Level 3 या एका गोष्टीवर उभा आहे की साखळी एकदा design करा, मग ती माणसाशिवाय चालते. अनिलची नळ-योजना आठवा (Chapter 0.2): हे तेच आहे, फक्त पाण्याऐवजी वीज वाहते.

Chapter 1.2 [SPINE]: मी हे गृहीत धरून चाललोय

प्रत्येक पुस्तक काही गोष्टी गृहीत धरतं आणि सांगत नाही. मग वाचणारा कुठेतरी अडतो आणि त्याला वाटतं आपणच कमी आहोत. चूक. पुस्तकाने गृहीत धरलं होतं. हे पुस्तक तसं करणार नाही. मी काय-काय गृहीत धरतोय त्याची ही पूर्ण यादी. जे माहीत आहे त्यावर खूण करा; जे नाही, त्याचं एका ओळीचं उत्तर इथेच आहे.

वीज म्हणजे काय? धातूच्या आत छोटे कण (electrons) असतात, जे धक्का मिळाल्यावर एका दिशेला सरकतात. त्या सरकण्याचं नाव current. पाण्याच्या pipe मधलं पाणी असा विचार करा, काम चालेल. (ही उपमा आहे, पूर्ण सत्य नाही; पण या पुस्तकासाठी पुरेशी आहे.)

तार current का नेते आणि plastic का नाही? काही वस्तूंमध्ये कण सहज सरकतात (धातू: **conductor**), काहींमध्ये अजिबात नाही (plastic, rubber: **insulator**). म्हणून तार धातूची आणि तिचं आवरण plastic चं: आत रस्ता, बाहेर भिंत. रस्ता आणि रस्त्याच्या कडेचा कठडा.

Battery काय करते? धक्का देते. तिची दोन टोकं असतात, आणि ती कणांना एका टोकाकडून दुसऱ्याकडे ढकलते. जोपर्यंत रस्ता जोडलेला आहे, धक्का चालू. हातपंपासारखी: दांडा मारत राहा, पाणी फिरत राहतं.

”Signal” म्हणजे काय? Current चं असणं किंवा नसणं, बस. पुढे जेव्हा म्हणू “phone ने signal पाठवला,” तेव्हा अर्थ: कुठल्यातरी तारेत किंवा हवेत current/लहरीचा एक क्रम गेला: आहे, नाही, आहे, आहे, नाही. Post office ची जुनी tar (telegraph) हेच करायची: टक-टक चा क्रम, आणि पलीकडे अर्थ.

Chip म्हणजे काय? Silicon च्या छोट्या तुकड्यावर छापलेले billions transistors आणि त्यांच्यामधल्या तारा, एका डबीत बंद. Computer च्या आतल्या हिरव्या पट्टीवर (circuit board) याच डब्या बसवलेल्या असतात.

बस. या पुस्तकाला विजेबद्दल यापेक्षा जास्त काहीही लागत नाही. वरच्या पाच गोष्टी ठीक वाटत असतील, तर पूर्ण पुस्तकासाठी पुरेसं physics तुमच्याकडे आहे.

इथे लोक काय चुकीचं समजतात

लोकांना वाटतं technology समजण्यासाठी आधी physics चा डोंगर चढावा लागेल, मग maths चा. नाही. प्रत्येक मजला खालच्या मजल्याचा फक्त छोटा हिस्सा वापरतो. Bus driver ला engine ची पूर्ण रचना लागत नाही; त्याला एवढंच लागतं: accelerator, brake, आणि हे की गाडी भरवशाची आहे.

Computer बनवणाऱ्यालाही electron ची पूर्ण physics लागत नाही; त्याला एवढंच लागतं: switch उघडतो, बंद होतो, भरवशाचा आहे. ही गोष्ट पुढे सगळीकडे लागेल: **प्रत्येक मजला खालच्याचं फक्त handle धरतो, पूर्ण engine नाही.** (हे abstraction आहे, Chapter 0.4 वालं.)

MAP वर

इथे कोण कमावतं: जे हा खालचा मजला पूर्ण जाणतात: electrical engineers, chip designers. त्यांची दुनिया या पुस्तकाच्या बाहेर आहे, आणि तिथेही billions चा बाजार आहे (power grids, batteries, solar). आपण त्या दरवाजासमोरून जातोय, आत घुसत नाही. लक्षात एवढंच ठेवा की तो दरवाजा आहे, आणि त्याच्या आतही Level 3 च आहे: कुठल्यातरी business ची ताकद वाढवणं.

स्वतः बघा (5 मिनिटं)

तुमचा phone charger बघा. त्यावर बारीक लिहिलेलं असेल: “5V 2A” सारखं काही. V धक्क्याचं माप आहे (किती जोरात), A प्रवाहाचं माप (किती कण प्रति second). आता charger ची तार बघा: आत धातू, बाहेर plastic. Conductor, insulator. तुमच्या हातात पूर्ण Chapter 1.2 आहे.

विचार करा

1. (derivation) पाण्याची उपमा कुठे तुटू शकते? एक जागा विचार करा जिथे वीज पाण्यासारखी नाही.

उत्तर: बऱ्याच जागी. सगळ्यात मोठी: पाणी pipe मधून गळालं तर फरशी ओली होते; पण current ला पूर्ण जोडलेला रस्ता लागतो, परत जाण्याच्या रस्त्याशिवाय current वाहतच नाही. म्हणून चिमणी एका तारेवर बसू शकते (रस्ता पूर्ण झाला नाही), पण दोन तारांना एकदम शिवली तर रस्ता पूर्ण होतो. उपमा हत्यार आहे, सत्य नाही. हा धडा AI पर्यंत जाईल: प्रत्येक model, प्रत्येक उपमा कुठेतरी तुटते, आणि expert तोच जो जाणतो कुठे.

Chapter 1.3 [SPINE]: Computer खरं आहे तरी काय

आतापर्यंतचे तुकडे जोडा: switch हो/नाही ठेवतो, switch switch ला दाबतो, साखळी आधीच design होते. मग ही पूर्ण गोष्ट मिळून करते काय?

एका कामापासून सुरुवात करा जे तुम्ही स्वतः करता: $47 + 38$ जोडणं. तुम्ही काय करता? तुमच्याकडे एक **पद्धत** आहे, लहानपणी पाठ केलेली: आधी एक जोडा, $7 + 8 = 15$, 5 लिहा, 1 हातचा. मग दहं जोडा, हातचा 1 पण. उत्तर 85. लक्ष द्या: तुम्ही **विचार** केला नाही, तुम्ही एक पाठ केलेली पद्धत **चालवली**. कुठलेही दोन आकडे द्या, पद्धत तीच राहील.

आता विचारा: switches च्या साखळीतून अशी पद्धत चालू शकेल का? हो. आकडे हो/नाही मध्ये ठेवता येतात (पुढचा chapter). जोडण्याची पद्धत switches च्या साखळीत आधीच design करता येते (Chapter 1.5). मग एक डबा बनू शकतो: या बाजूने दोन आकडे घाला, त्या बाजूने बेरीज निघेल. कुठल्याही माणसाशिवाय.

आणि आता खरी उडी. जोडण्याचा डबा बनू शकतो, तर: तुलना करण्याचा डबा (कुठला मोठा?), copy करण्याचा डबा, “जर हे तर ते” चा डबा. आणि हे डबे रांगेत लावले, एकामागून एक, तर लांबलचक recipe चालू शकते: “आधी जोडा, मग तुलना करा, मोठं असेल तर हे copy करा...”

मग computer हे आहे: एक machine जी आकड्यांवर, आधीच लिहिलेली recipe, न थकता, billion पावलं प्रति second च्या वेगाने चालवते.

या व्याख्येतले तीन शब्द नीट बघा. **आकड्यांवर**: computer ला फक्त numbers कळतात; photo, गाणं, message, सगळं आधी number बनेल (Section 2 चं काम). **आधीच लिहिलेली**: computer च्या प्रत्येक पावलामागे कुठल्यातरी माणसाचा आधीच घेतलेला निर्णय आहे; machine निर्णय घेत नाही, निर्णय **चालवते**. **न थकता**: हीच तिची पूर्ण किंमत; recipe एकदा लिहा, machine billion वेळा चालवेल. Leverage, शुद्ध रूपात.

या recipe चं नाव **program**. Recipe लिहिण्याचं नाव **programming**. सगळ्या recipes मिळून **software**, आणि हाताला लागणारी machine **hardware**. जोडणारा-तुलना करणारा मुख्य डबा **processor** किंवा **CPU**.

ढाब्याच्या भाषेत: मालकीण एकदा recipe लिहून देते; नवा cook रोज तीच चालवतो, विचार न करता, आणि चव तीच येते. Recipe लिहिणं वेगळं काम, चालवणं वेगळं. आणि खऱ्या जगात हे असं दिसलं: bank चा clerk 1960 मध्ये interest मोजायचा: प्रत्येक खात्यावर तीच पावलं, दिवसभर, चुकांसह. आज

तीच recipe software मध्ये लिहिलेली आहे आणि रात्री 2 वाजता कोट्यवधी खात्यांवर एकदम चालते, शून्य थकवा. Clerk ची पद्धतच machine मध्ये गेली. माणसाचं काम recipe लिहिणं उरलं, चालवणं machine घेऊन गेली. जिथे जिथे जगात एखादी “पाठ केलेली पद्धत” पुन्हा पुन्हा चालते, तिथे तिथे computer घुसेल. ही ओळ business शोधण्याचं हत्यार आहे, जपून ठेवा.

इथे लोक काय चुकीचं समजतात

“Computer विचार करतो.” नाही. तो recipe चालवतो. Computer “चूक” करतो तेव्हा ती recipe ची चूक असते, कुठल्यातरी माणसाने लिहिलेली. Computer “ठरवतो” तेव्हा तो माणसाचा आधीच लिहिलेला निर्णय असतो. ही गोष्ट AI पर्यंतही जाईल; तिथे recipe विचित्र पद्धतीने लिहिली जाते, पण machine तीच आहे. ज्याला हे समजलं, तो machine ला कधी घाबरणार नाही आणि तिच्यावर कधी आंधळा विश्वासही ठेवणार नाही.

MAP वर

या chapter ने Level 3 ला दोन भागांत कापलं: hardware आणि software. दोघांचे बाजार वेगळे. Hardware मध्ये पैसा factory आणि scarcity तून येतो (TSMC, Samsung, Nvidia). Software मध्ये पैसा या गोष्टीतून येतो की लिहिण्याचा खर्च **एकदाच** लागतो: recipe एकदा लिहिली, आता प्रत्येक copy फुकट. म्हणून जगातल्या सगळ्यात जाड नफ्याच्या companies software वाल्या आहेत: Microsoft चा माल बनवण्याचा खर्च जवळजवळ शून्य आहे. तुम्ही जे शिकताय, ते याच बाजूचं दार आहे.

स्वतः बघा (5 मिनिटं)

Phone चा calculator उघडा. $47 + 38$ करा. आता स्वतःला विचारा: या काचेखाली काय झालं? तुमचे taps numbers बनले, numbers CPU च्या जोडणाऱ्या डब्यात गेले, उत्तराचे numbers screen चे ठिपके बनले. पूर्ण Chapter 1.3 एका second मध्ये चाललं. आणि ही calculator-recipe कोणी तरी **एकदा** लिहिली होती; आता billions phones वर फुकट चालते.

विचार करा

1. (derivation) “Machine निर्णय घेत नाही, निर्णय चालवते.” मग bank चं software तुमचं loan नाकारतं तेव्हा खरा निर्णय कोणी घेतला? आणि या उत्तरातून business ची एक जबाबदारी काढा.

उत्तर: निर्णय त्या माणसाने घेतला ज्याने recipe लिहिली किंवा लिहून घेतली: “जर income इतक्यापेक्षा कमी, तर नाही.” Machine ने फक्त तो न थकता चालवला. जबाबदारी: business जेव्हा आपला निर्णय recipe मध्ये घालतो, तेव्हा चुकीलाही leverage मिळतो; एक चुकीचा नियम लाखो लोकांवर एकदम चालतो. म्हणून recipe लिहिणाऱ्याची किंमतही जास्त आहे आणि जबाबदारीही. AI च्या जमान्यात हा प्रश्न अजून मोठा होईल; Book 2 मध्ये भेटेल.

Chapter 1.4 [SPINE]: Switches आकडे कसे ठेवतात

एक switch दोन गोष्टी सांगू शकतो: 0 किंवा 1. पण जगातले आकडे तर 0 ते 9 च्या अंकांनी बनतात, आणि मोठे आकडे, जसे 300 रुपये किंवा 15 billion, त्यांचं काय?

स्वतः काढू. एक switch: दोन स्थिती (0, 1). दोन switch: चार स्थिती (00, 01, 10, 11). तीन switch: आठ. प्रत्येक नवा switch स्थितींची मोजणी **दुप्पट** करतो, कारण जुनी प्रत्येक स्थिती आता दोन रूपांत असू शकते: नवा switch 0 सोबत, किंवा 1 सोबत.

आता या स्थितींना नावं द्या, एवढंच करायचं आहे:

00 ला म्हणा "शून्य"	01 ला म्हणा "एक"
10 ला म्हणा "दोन"	11 ला म्हणा "तीन"

थांबा, हे तर तेच आहे जे आपण दहा बोटानी करतो! आपण 9 नंतर नवा अंक सुरू करतो (10), machine 1 नंतर नवा switch सुरू करते (10). आपला प्रत्येक अंक दहापट वाढतो, machine चा प्रत्येक switch दुप्पट. बस, एवढाच फरक. मोजणीची कल्पना तीच, फक्त बोटं 10 ऐवजी 2.

एका switch च्या स्थितीचं नाव **bit** (binary digit). आठ bits च्या गटाचं नाव **byte**. दुप्पट-वाल्या मोजणीचं नाव **binary**. हे तीन शब्द आता तुमचे झाले, आणि पूर्ण digital जग यांनीच मोजलं जातं.

मग कामाच्या आकड्यांसाठी किती switch लागातात?

8 switch	=	256 स्थिती	(0 ते 255)
16 switch	=	65,536	
32 switch	=	4 billion+	(430 कोटी)
64 switch	=	इतकं मोठं की जगातले सगळे रुपये, सगळे phones, सगळ्या files मोजा, संपत नाही	

तुमच्या phone चा processor एकदम 64 switches च्या गटावर काम करतो. म्हणून खोक्यावर “64-bit phone” लिहिलेलं असतं. आता तुम्हाला त्या ओळीचा अर्थ माहीत आहे; जगातल्या 99% लोकांना नाही.

एक शेवटचा प्रश्न स्वतःला विचारा: machine 2 वर का थांबली, 10 स्थितीवाला switch का नाही बनवला? बनू शकला असता. पण विजेचा current थोडा वर-खाली होत राहतो (noise). दोन स्थितींमध्ये फरक इतका मोठा आहे (पूर्ण चालू, पूर्ण बंद) की छोट्या-मोठ्या कंपनेने चूक होत नाही. दहा स्थिती जवळजवळ

असत्या, आणि प्रत्येक हलकी कंपनीची चुकीचा अंक बनली असती. Post office चा शिक्का आठवा: तारीख थोडी तिरकी उमटली तरी चालतं, कारण पर्याय मोजके आणि एकमेकांपासून दूर आहेत. **2 निवडला कारण 2 सगळ्यात भरवशाचा आहे.** Machine च्या प्रत्येक design मध्ये हा प्रश्न भेटेल: वेगवान हवं, स्वस्त हवं, की भरवशाचं हवं? इथे भरवसा जिंकला.

खऱ्या जगात हे कुठे आहे: तुमचा bank balance कुठल्यातरी machine मध्ये 64 switches ची एक रांग आहे. UPI ने 50 रुपये पाठवले तर त्या रांगेतले काही switches पलटले, आणि दुसऱ्या machine मध्ये दुसऱ्या रांगेतले. पूर्ण Indian economy, प्रत्येक खातं, प्रत्येक देवाण-घेवाण, चालू/बंद च्या रांगांमध्ये ठेवलेली आहे. म्हणून त्या रांगांची राखण (Part 3 मधलं encryption) इतका मोठा business आहे: switches मध्ये ठेवलेला पैसा खरा पैसा आहे.

इथे लोक काय चुकीचं समजतात

लोक binary बघून विचार करतात की machine कुठलीतरी वेगळी, परकी भाषा बोलते जी माणसाच्या आवाक्याबाहेर आहे. तुम्ही आता 5 मिनिटांत पूर्ण कल्पना काढली, कारण नवीन काही नव्हतंच: ही तीच मोजणी आहे जी तुम्ही लहानपणापासून करता, कमी बोटांवर. Technology मध्ये “अवघड” हे बहुतेक वेळा “अनोळखी” चं दुसरं नाव असतं.

MAP वर

रुपयाचा रस्ता: तुम्ही जे काही साठवता ते bits मध्ये मोजलं जातं, आणि bits ठेवायच्या जागेचं भाडं लागतं. गोदामासारखं: पोती जास्त, भाडं जास्त. Google तुम्हाला 15 GB फुकट देतो आणि त्यावर महिन्याचं भाडं घेतो (Google One, iCloud). ते भाडं billions चा business आहे. पुढे Chapter 2.1 मध्ये हे माप पूर्ण शिकू, कारण जे मोजलं जातं, तेच विकलं जातं.

स्वतः बघा (5 मिनिटं)

एका हाताची पाच बोटे घ्या. मिटलेलं बोट 0, उघडं 1. अंगठ्यापासून सुरू करा: फक्त अंगठा उघडा = 1, फक्त दुसरं बोट = 2, दोन्ही = 3. असं मोजत जा. पाच बोटांवर तुम्ही 31 पर्यंत मोजू शकता. हे एकदा केलंत की binary हाताने चालवली; आता ती कधीच विसरणार नाही.

विचार करा

1. (derivation) 8 switches 256 स्थिती देतात. एक जुना game console “8-bit” होता: screen वरचा एक रंग निवडायला 8 bits. त्या console वर किती रंग शक्य होते, आणि यातून काढा: जुने games तसे का दिसायचे?

उत्तर: 256 रंग, बस. म्हणून जुन्या games चे रंग स्पष्ट तुकड्यांत दिसतात; संध्याकाळच्या आभाळाचा हळू बदलणारा रंग त्यांच्यात बनूच शकत नव्हता. आजचे screens प्रत्येक ठिपक्यासाठी 24+ bits ठेवतात, 16 million (1.6 कोटी) रंग, म्हणून photo खरा वाटतो. धडा: bits ची मोजणी थेट ठरवते की machine किती बारकावा बघू शकते. हा नियम AI पर्यंत जाईल: तिथेही बारकावा bits मध्ये विकला जातो.

Chapter 1.5 [DEPTH]: Switches

ठरवतात कसे

(हा DEPTH chapter आहे. पहिल्या वेळेस जड वाटला तर पुढे जा, Part 1 संपवून परत या. पण परत नक्की या, कारण इथेच “machine विचार करते” वाली जादू पूर्ण तुटते.)

Chapter 1.3 मध्ये आपण मानलं होतं की “जोडण्याचा डबा” आणि “तुलनेचा डबा” बनू शकतात. आता बनवू. फक्त तीन छोटे भाग लागतात, तिन्ही switches पासून बनतात. दरवाजांचा विचार करा:

AND: दोन दरवाजे एकामागून एक, रांगेत. पलीकडे जायचं तर **दोन्ही** उघडे लागतील. म्हणजे: “दोन्ही हो, तर हो.” सरकारी कामासारखं: form वर दोन साह्या लागतात, एक जरी नसली तरी file पुढे जात नाही.

OR: दोन दरवाजे शेजारी-शेजारी, दोन वेगळे रस्ते. **कुठलाही एक** उघडा असला तरी पलीकडे जाता येतं. म्हणजे: “कोणीही हो, तर हो.” गावात जायला दोन रस्ते: पूल बंद असला तरी वळणाचा रस्ता चालतो.

NOT: उलट करणारा. आत 1 तर बाहेर 0, आत 0 तर बाहेर 1.

बस. हे तीनच आहेत. यांचं नाव **logic gates**. आता यांच्याकडून काम करून घेऊ.

तुलना बनवा: “A आणि B वेगळे आहेत का?” विचार करा: वेगळे म्हणजे (A हो, B नाही) **किंवा** (A नाही, B हो). हे तर AND, OR, NOT ची जोडणी आहे: (A AND (NOT B)) OR ((NOT A) AND B). झालं. Machine आता “वेगळं आहे का” सांगू शकते. आणि “सारखं आहे का?” त्याचं NOT आहे.

बेरीज बनवा: एक bit जोडा: $0+0=0$, $0+1=1$, $1+0=1$, $1+1=10$ (दोन bits चं उत्तर: “0 लिहा, 1 हातचा”). बघा, उत्तराचा पहिला भाग “वेगळे आहेत?” वालाच भाग आहे, आणि हातच्याचा भाग AND आहे (दोन्ही 1 असतील तेव्हाच हातचा जातो). शाळेतली बेरजेची पद्धत switches मध्ये उतरली. असे 64 जोडा, आणि 64-bit आकड्यांची बेरीज तयार.

”जर” बनवा: “जर हे 1 असेल तर या तारेचा signal पुढे जाऊ दे.” हे फक्त एक AND आहे: signal AND अट. Recipe चं प्रत्येक “जर असं तर तसं” शेवटी याच AND/OR/NOT च्या साखळ्या आहेत. ATM चा निर्णय बघा: “पैसे द्या **जर** PIN बरोबर **AND** balance पुरेसा **AND** आजची limit शिल्लक.” तीन अटी, एक AND-साखळी. तुम्ही ATM मधून पैसे काढता तेव्हा gates चा एक गट तुमच्या बाजूने 1 म्हणतो.

आता पूर्ण मनोरा खालून वर बघा:

transistor -> AND/OR/NOT -> बेरीज/तुलना/जर -> recipe ची पावलं
 -> पूर्ण program -> WhatsApp

प्रत्येक मजला खालच्या मजल्याच्या भागांनी बनला आहे. कुठेही, कुठल्याही मजल्यावर, “विचार” नावाचा भाग नाही. फक्त switches आहेत, design मध्ये जडवलेले.

इथे लोक काय चुकीचं समजतात

”Machine ठरवते, म्हणजे तिच्या आत कुठेतरी छोटा मेंदू आहे.” आता तुम्हाला “ठरवणं” चा अर्थ माहीत आहे: आधीच design केलेली AND/OR/NOT ची साखळी, जिच्यात प्रत्येक शक्य input चं output **आधीच** ठरलेलं आहे. Machine कडे surprise नसतं. Machine चं उत्तर तुम्हाला surprise करतं, तेव्हा अर्थ: तुम्हाला design माहीत नाही; machine काही नवीन विचार करत नाहीये.

MAP वर

कोण कमावतं: chip design करणारे. Gates जोडून वेगवान, छोटी, कमी वीज खाणारी साखळी बनवणं जगातल्या सगळ्यात दुर्मिळ skills पैकी आहे. **Nvidia** याच खेळात जगातल्या सगळ्यात मौल्यवान companies मध्ये पोहोचली: त्यांच्या gates ची जुळणी AI च्या कामासाठी सगळ्यात वेगवान निघाली. जी गोष्ट तुम्ही आज 15 मिनिटांत समजलात, तिचीच उत्तम जुळणी trillion-dollar company बनवते. फरक खोलीचा आहे, कल्पनेचा नाही.

स्वतः बघा (5 मिनिटं)

कागदावर हे table स्वतः बनवा, न बघता: A आणि B च्या चार जोड्या (00, 01, 10, 11), आणि प्रत्येक जोडीसाठी AND, OR, आणि “वेगळे आहेत?” चं उत्तर. शेवटचा रकाना स्वतः विचार करावा लागेल. बनला की समजा: आज तुम्ही gate design केला.

विचार करा

1. (derivation) NOT gate काढून टाका. फक्त AND आणि OR उरले. “वेगळे आहेत?” वाला भाग अजूनही बनू शकेल का? का किंवा का नाही?

उत्तर: नाही बनू शकत. AND आणि OR दोन्ही “मिळवणारे” आहेत: सगळे inputs 1 असतील तर दोघांचंही output 1 च असतं; उलटवणारा कोणी नाही. पण “वेगळे आहेत?” ला 11 वर 0 द्यावं लागतं. उलटवणाऱ्या भागाशिवाय 1 पासून 0 बनतच नाही. म्हणून NOT गरजेचा आहे. धडा: छोटासा भागही पूर्ण system ची ताकद बदलतो, आणि design म्हणजे हेच जाणणं की कुठला भाग काय जोडतो.

Chapter 1.6 [SPINE]: एकच machine सगळं का करू शकते

एका विचित्र गोष्टीकडे कधी लक्ष दिलंय? Mixer फक्त वाटतो. Fridge फक्त थंड करतो. गाडी फक्त चालते. प्रत्येक machine **एक** काम करते.

पण तुमचा phone? सकाळी alarm, मग camera, मग bank, मग game, मग video call. एकच machine, हजार कामं. जगातल्या बाकी प्रत्येक machine पेक्षा वेगळी. असं काय आहे हिच्यात?

Chapter 1.3 मधून recipe ची गोष्ट उचला. Machine recipe चालवते. आता एक प्रश्न, जो पूर्ण इतिहासातला सगळ्यात सुंदर प्रश्न आहे:

Recipe स्वतः काय आहे?

Recipe म्हणजे पावलांची यादी: “जोडा,” “तुलना करा,” “जर असं तर तिकडे जा.” आता या पावलांना नंबर द्या: जोडा = 1, तुलना = 2, copy = 3... मग पूर्ण recipe बनली numbers ची रांग: 1, 3, 2, 7...

थांबा. Numbers ची रांग? ती तर machine switches मध्ये ठेवू शकते (Chapter 1.4). म्हणजे:

Recipe त्याच जागी ठेवता येते जिथे data ठेवला जातो. आणि machine अशी बनवा की ती आपल्या memory मधून पुढचा number वाचेल, आणि त्या number चं काम करेल. बस.

याचा अर्थ उलगाडा, कारण हा Level 3 चा जन्म आहे:

नवं काम हवं? नवी machine बनवू **नका**. फक्त memory मध्ये नवी recipe घाला. Machine तीच, काम नवं. Camera app घातलं तर phone camera झाला, bank app घातलं तर bank झाला. Machine बदलली नाही, फक्त numbers बदलले.

ढाब्याची उपमा परत: mixer म्हणजे एकाच पदार्थाचा cook. पण एक cook जो **कुठलीही** लिहिलेली recipe वाचून बनवू शकतो, तो एकटा पूर्ण hotel आहे. आज पंजाबी, उद्या चायनीज; cook तोच, कागद बदलला. तुमचा phone हा तो cook आहे. या design चं नाव **stored-program computer** (recipe data च्या जागी ठेवणं), आणि “एक machine जी कुठलीही recipe चालवू शकते” या कल्पनेचं नाव **universal machine**. Alan Turing ने 1936 मध्ये, machine बनण्याआधी, कागदावर हे सिद्ध केलं होतं.

आणि आता पैसा बघा. Machine बनवणं: factory, सामान, मजूर, प्रत्येक copy चा खर्च. Recipe बनवणं: एकदा डोकं लावा, मग प्रत्येक copy **जवळजवळ फुकट**, कारण copy म्हणजे फक्त numbers ची नक्कल. Chapter 0.2 चा leverage चा काटा आठवा. Software ही पहिली गोष्ट आहे जिथे leverage चा काटा टोकापर्यंत फिरतो: एकदा लिहा, billion वेळा चालवा, copy चा भाव शून्य.

याच एका कल्पनेने जगाच्या श्रीमंतीचा नकाशा बदलला. **Nokia** जगातली सगळ्यात मोठी phone company होती. तिचे phones **एका** कामासाठी बनले होते: call आणि SMS. iPhone आला आणि त्याने phone ला universal machine बनवलं: app store म्हणजे “कुठलीही recipe घाला.” Nokia hardware चांगलं करत राहिली; Apple ने recipes चा बाजार नेला. दहा वर्षांत Nokia संपली. Machine विरुद्ध universal machine च्या लढाईत universal नेहमी जिंकते, कारण तिच्याकडे जगभरच्या recipe लिहिणाऱ्यांची फौज असते.

इथे लोक काय चुकीचं समजतात

“Software company काढायला factory सारखा मोठा पैसा लागतो.” उलटं. Software ची factory नाहीच, म्हणून सुरुवातीचा खर्च इतिहासातला सगळ्यात कमी आहे: एक laptop, एक डोकं. म्हणूनच दोन मुलं garage मध्ये Google सुरू करू शकली. आणि म्हणूनच स्पर्धाही भयंकर आहे: जी गोष्ट तुम्ही स्वस्तात सुरू करू शकता, ती कोणीही स्वस्तात सुरू करू शकतो. Scarcity आता recipe च्या **कल्पनेत** आणि **quality** त आहे, सुरू करण्यात नाही.

MAP वर

India चं उदाहरण: **Zerodha**. दोन भावांनी stock broking ची recipe software मध्ये लिहिली. Branch नाही उघडली, agents नाही ठेवले. एक recipe, 10 million+ (1 कोटी+) ग्राहक, आणि India च्या सगळ्यात नफ्यातल्या startups पैकी एक, बाहेरचा पैसा न घेता. Universal machine वर लिहिलेली एक चांगली recipe काय करू शकते, याचा देशी पुरावा. तुमचा Level 4 वाला plan याच जमिनीवर उभा राहील.

स्वतः बघा (5 मिनिटं)

Phone च्या Settings मध्ये जा, Storage बघा. तिथे apps ची यादी असेल, प्रत्येकीपुढे MB/GB. आता विचार करा: प्रत्येक ओळ एक recipe आहे, numbers ची रांग, जी याच एका machine ला शंभर वेगळ्या machines बनवते. WhatsApp ~100 MB ची recipe आहे. तिची प्रत्येक copy फुकट बनते, आणि 3 billion (300 कोटी) लोक चालवतात. आता सूत्र स्वतः लावा: $\text{size} \times \text{scarcity} \times \text{leverage}$.

विचार करा

1. (derivation) Recipe data च्या जागी राहते. मग एक recipe दुसऱ्या recipe ला **वाचू** किंवा **लिहू** शकते, कारण ती फक्त numbers आहे. या गोष्टीमुळे एक खूप मोठी गोष्ट शक्य होते जी तुम्ही रोज वापरता. कुठली?

उत्तर: Update. WhatsApp स्वतःला update करतं तेव्हा एक recipe (updater) दुसऱ्या recipe ला (जुनं WhatsApp) नव्या numbers नी बदलत असते. Machine ला हात न लावता, जगभरच्या machines रातोरात नवं काम शिकतात. आणि याच दरवाजाने virus पण घुसतो: तोही एक recipe आहे जी दुसऱ्या recipes ना लिहिण्याचा प्रयत्न करते. एकच ताकद, चांगल्या आणि वाईट दोन्ही हातांत. Part 3 मध्ये हिची राखण भेटेल.

SECTION 2: NUMBERS आणि SIZE

पाच chapters. Section 1 ने दाखवलं की machine numbers वर recipe चालवते. आता दोन प्रश्न उरले: जगातल्या गोष्टी (text, photo, आवाज) numbers कशा बनतात, आणि numbers किती जागा घेतात.

हे तुमच्या business decision मध्ये कुठे येईल: प्रत्येक tech product चा खर्च तीन गोष्टींनी बनतो: data किती मोठा, किती वेगवान हवा, किती साठवायचा. हा section तुम्हाला ते माप देतो. ज्या founder ला GB आणि millisecond चा अर्थ माहीत आहे, तो engineer च्या estimate वर प्रश्न विचारू शकतो आणि cloud चं bill वाचू शकतो. जो जाणत नाही, तो अंधारात सही करतो.

Chapter 2.1 [SPINE]: एक गोष्ट किती जागा घेते

कापड meter मध्ये मोजतात, दूध litre मध्ये, आणि digital गोष्टी? नावं तुम्ही ऐकली आहेत: MB, GB. आता ती कमावण्यालायक समजून घेऊ.

मापाचं एकक तेच आहे जे Chapter 1.4 मध्ये बनलं: **bit**, एका switch ची स्थिती. आठ bits चा गट **byte**. एका byte मध्ये 256 स्थिती मावतात; एक अक्षर ठेवायला एवढं पुरे. मग एक ढोबळ गोष्ट पकडा:

1 byte = साधारण 1 अक्षर.

आता सरळ मोजणी, प्रत्येक पायरीवर हजाराचा गुणाकार:

1 KB (kilobyte)	= ~1 हजार bytes	= एक परिच्छेद
1 MB (megabyte)	= ~1 million bytes	= एका जाड पुस्तकाचा text
1 GB (gigabyte)	= ~1 billion bytes	= 1,000 जाड पुस्तकं
1 TB (terabyte)	= ~1 trillion bytes	= एक छोटी library

आता खऱ्या जगाचे anchors; हे पाठ करण्यालायक आहेत, कारण प्रत्येक बोलण्यात येतील:

WhatsApp message (फक्त text)	~ 100 bytes
एक photo (phone camera)	~ 3 MB
एक गाणं	~ 5 MB
एक तास video call	~ 500 MB - 1 GB
एक movie (चांगली quality)	~ 2-4 GB
तुमच्या phone चा पूर्ण storage	~ 128-256 GB

एक pattern दिसतोय? Text चिमूटभर आहे. Photo त्याच्या हजारपट. Video photo च्याही शंभरपट.
Text : photo : video = 1 : हजार : लाख. हे प्रमाण या पूर्ण पुस्तकातला सगळ्यात कामाचा आकडा आहे, कारण जिथे size आहे तिथे खर्च आहे.

असं का? Text मध्ये प्रत्येक अक्षर एक byte. Photo मध्ये प्रत्येक ठिपक्याचे (pixel) तीन bytes, आणि ठिपके millions मध्ये. Video मध्ये प्रत्येक second ला 30 photos. मोजणी स्वतःच गुणाकार होत जाते. बाजाराच्या भाषेत: चिठ्ठी, पोतं आणि truck. चिठ्ठी खिशात मावते, पोत्याला हमाल लागतो, truck ला रस्ता आणि diesel.

एक बारीक गोष्ट, जिच्यात दुकानदाराची चलाखी लपली आहे: internet चा वेग **bits** मध्ये मोजला जातो (Mbps = megabits per second), storage **bytes** मध्ये (MB). Bit हा byte चा आठवा हिस्सा. म्हणून “100 Mbps” चं connection एका second मध्ये फक्त 12.5 MB आणतं. Telecom companies bits मध्ये बोलतात कारण आकडा आठपट मोठा दिसतो. आता तुम्ही ही चाल पकडू शकता.

आणि हे कमाईशी असं जोडलेलं आहे: WhatsApp वर पाठवलेला photo हलका का होतो? कारण WhatsApp तो पाठवण्याआधी दाबतं (compress): 3 MB वरून ~100 KB, तीसपट छोटा. रोज billions photos जातात; full size गेले असते तर WhatsApp चा data खर्च तीसपट असता, आणि तुमचा data pack पण. एका compression recipe ने company चे billions वाचवले. Size ची व्यवस्था हाच पैसा आहे.

इथे लोक काय चुकीचं समजतात

लोक KB, MB, GB ला नुसतं “छोटा, मोठा, अजून मोठा” समजतात. नाही: प्रत्येक पायरी **हजाराचा** गुणाकार आहे. MB आणि GB मधला फरक तोच आहे जो 1 रुपया आणि 1,000 रुपयांमधला. Engineer म्हणतो “ही file GB मध्ये आहे,” तेव्हा तो म्हणतोय “ही MB वालीपेक्षा हजारपट जड आहे,” आणि तिचा खर्चही तसाच असेल.

MAP वर

रुपयाचा रस्ता: तुम्ही Google ला storage चं भाडं देता (Google One: 100 GB साठी महिना ~130 रुपये). Google तो data ज्यांच्या machines वर ठेवतो त्यांच्या disks Samsung/Seagate कडून येतात. खाली उतरत जा, एक गरज मिळेल: “माझ्या आठवणी हरवू नयेत” (Level 1). त्या गरजेचा पूर्ण stack: Photos app (L2 सेवा), cloud storage (L3), disk factory (L3 चा hardware कोपरा). प्रत्येक GB भाडं देतोय.

स्वतः बघा (5 मिनिटं)

Phone मध्ये एक photo उघडा, त्याची details/info बघा (size मिळेल, ~2-4 MB). आता एका text message चा size आठवा (~100 bytes). आता स्वतः मोजा: एक photo किती messages च्या बरोबर आहे? (~30,000.) आता समजेल की data pack video बघण्यानेच का संपतो, text ने कधीच नाही.

विचार करा

1. (derivation) दोन startup कल्पना: एकीत लोक 200 शब्दांचे reviews लिहितात, दुसरीत लोक 1 मिनिटाचे video reviews बनवतात. दोन्हीकडे 1 million users. फक्त size च्या ज्ञानाने सांगा: कोणाचा खर्च किती जास्त, आणि याचा business वर काय परिणाम?

उत्तर: Text review ~1 KB, video review ~50-100 MB: फरक जवळजवळ **लाखपट**. मग video वाल्याचा साठवण्याचा आणि पाठवण्याचा खर्चही हजारो-लाखोपट. परिणाम: text वालं app फुकट users वरही जगू शकतं; video वाल्याला पहिल्या दिवसापासून पैशाचा रस्ता लागतो (ads, subscription), नाहीतर प्रत्येक नवा user त्याला गरीब करतो. Product चा आकार त्याच्या business model चा आकार ठरवतो. हे सूत्र आता तुमच्याकडे आहे; कुठल्याही कल्पनेवर लावा.

Chapter 2.2 [SPINE]: Text, photo, आवाज, video: सगळे numbers

Machine ला फक्त numbers कळतात. मग तुमचा लग्नाचा photo, आईच्या आवाजाची recording, या पुस्तकाचा text, हे सगळं machine मध्ये कसं घुसलं? प्रत्येकाचं उत्तर एकाच चालीने निघेल. चाल ही आहे:

गोष्टीचे छोटे-छोटे तुकडे करा, प्रत्येक तुकड्याला एक number द्या, आणि numbers चा अर्थ आधीच सगळ्यांनी मिळून ठरवा.

Pincode ची व्यवस्था आठवा: 400001 म्हणजे Mumbai GPO, हे कुठल्या कागदात लपलेलं सत्य नाही; post खात्याने **ठरवलं** आणि सगळ्यांनी मानलं. आता हीच चाल चारही गोष्टींवर लावा:

Text. तुकडा = अक्षर. सगळ्यांनी मिळून ठरवलं: A = 65, B = 66, space = 32. मग "HI" machine मध्ये 72, 73 आहे. बस. तुमच्या नावाचं spelling numbers ची रांग आहे. या ठरवलेल्या table चं नाव **encoding**; जुनं table **ASCII** (1963), आजचं पूर्ण table **Unicode**, ज्यात जगातली प्रत्येक भाषा आहे: अ = 2309, आणि emoji सुद्धा (हसणारा चेहरा = 128514). तुम्ही वाचताय ती मराठी अक्षरंही याच क्षणी numbers म्हणूनच साठवलेली आहेत.

Photo. तुकडा = ठिपका (**pixel**). Photo ला जाळीत कापा, प्रत्येक चौकोनाचा एक रंग. आणि रंग? प्रत्येक रंग लाल, हिरवा, निळा यांच्या मिश्रणाने बनतो, म्हणून तीन numbers (0-255) प्रत्येक ठिपक्यासाठी: (255, 0, 0) = पूर्ण लाल. 4 million ठिपके x 3 numbers = तुमचा photo. रांगोळीसारखं: दुरून चित्र, जवळून फक्त ठिपके.

आवाज. आवाज म्हणजे हवेचं कंपन. तुकडा = वेळेचा क्षण. प्रत्येक second चे ~44,000 क्षण करा, प्रत्येक क्षणी कंपनाची उंची मोजा, तो number लिहा (एका मापाचं नाव **sample**). Numbers ची रांग परत वाजवा, आवाज परत. आईचा आवाज म्हणजे 44,000 numbers प्रति second.

Video. तुकडा = **frame**. प्रत्येक second ला 30 photos, अधिक आवाज. म्हणून video सगळ्यात जड: photo चा खर्च x 30 x seconds.

तीन पावलांमध्ये सगळ्यात महत्वाचं कुठलं? कापणं? मोजणं? नाही: "अर्थ आधीच सगळ्यांनी मिळून ठरवा." 72 चा अर्थ H तेव्हाच आहे जेव्हा पाठवणारा आणि घेणारा दोघे एकच table मानतात. जुन्या websites वर मराठी लिहिल्यावर ??? किंवा चौकोन दिसायचे, ते हेच: numbers पोहोचले, पण त्या

बाजूचं software Unicode चा मराठी भाग जाणत नव्हतं. Numbers बरोबर, अर्थ हरवला. ही गोष्ट, की **numbers चा अर्थ करार असतो, numbers मध्ये नसतो**, पुढे internet (Part 3) आणि AI (Book 2) दोन्हींचा पाया आहे.

इथे लोक काय चुकीचं समजतात

लोकांना वाटतं machine photo “बघते” किंवा गाणं “ऐकते”. नाही. Machine साठी photo म्हणजे 12 million numbers आणि गाणं म्हणजे 15 million. तिला ना रंग माहीत ना राग. आणि तरीही (हे पुढे Book 2 चं बीज आहे) numbers च्या pattern वरून machine सांगते की photo त मांजर आहे. बघणं नाही, मोजणीतून ओळख. जेव्हा ती गोष्ट येईल, तेव्हा हा chapter आठवा: सगळं numbers च होतं, नेहमी.

MAP वर

Company case: **JioCinema/Hotstar** cricket दाखवतात तेव्हा 40 million लोक एकाच वेळेस frames चे numbers मागत असतात. त्यांचा पूर्ण engineering खर्च याच chapter च्या प्रमाणातून येतो: video = सगळ्यात जड numbers. आणि त्यांचं business model पण तेच ठरवतं: ads आणि subscription गरजेचे, कारण प्रत्येक प्रेक्षकाचा data खर्च खरा आहे. Level 1 गरज: करमणूक. तिचे numbers: सगळ्यात महाग.

स्वतः बघा (5 मिनिटं)

कोणाला तरी हा message पाठवा: “72 69 76 76 79”. आणि सांगा: प्रत्येक number मधून 64 वजा करा आणि A=1, B=2 च्या मोजणीने अक्षर बनवा. उत्तर येईल HELLO. तुम्ही आत्ता स्वतःचं encoding table बनवलं आणि एका माणसाकडून decode करून घेतलं. Machine बस हेच करते, billion पट वेगाने.

विचार करा

1. (derivation) WhatsApp voice message पाठवताना “पाठवण्याआधी दाबा” (compress) कसं शक्य आहे? आवाज तर मोजलेलाच होता, 44,000 numbers प्रति second. कुठले numbers फेकता येतात?

उत्तर: ते, जे माणसाचे कान तसेही पकडत नाहीत. खूप उंच कंपन, मोठ्या आवाजामागे दबलेला हलका आवाज, दोन क्षणांमधला फरक जो अंदाजाने परत बनवता येतो. Recipe ते फेकून देते, size दहापट घटतो, कानाला फरक पडत नाही. धडा: compression म्हणजे “अर्थ वाचवा, numbers फेका.” आणि लक्ष द्या: काय फेकायचं हा एक **निर्णय** आहे, आणि प्रत्येक निर्णय कोणा माणसाच्या recipe त लिहिलेला आहे. Machine मध्ये सगळीकडे असे लपलेले निर्णय भेटतील.

Chapter 2.3 [SPINE]: मोठ्या आकड्यांचा अर्थ

या पुस्तकात आतापर्यंत आले: 15 billion transistors, 44,000 samples प्रति second, billion पावलं प्रति second. पुढे अजून मोठे येतील. Problem ही आहे की माणसाचं डोकं लाखाच्या पुढचे सगळे आकडे एकसारखे “खूप मोठे” मानतं. ते चालणार नाही, कारण या आकड्यांमधल्या **फरकातच** सगळी समज आणि सगळा पैसा आहे.

आधी भाषांतराचं table, कारण India मोजणी आपली बोलतो आणि technology इंग्रजी:

1 million	(M)	= 10 लाख
10 million		= 1 कोटी
1 billion	(B)	= 100 कोटी = 1 अब्ज
1 trillion	(T)	= 1 लाख कोटी

आता फरक जाणवून घ्या, घड्याळाने. एका second ला एक अशी मोजणी करा:

1 million पर्यंत मोजायला	~ 11 दिवस लागतील
1 billion पर्यंत	~ 31 वर्ष
1 trillion पर्यंत	~ 31,000 वर्ष

Million आणि billion “जवळजवळ” नाहीत. त्यांच्यामध्ये हजारपट अंतराचा समुद्र आहे. बातमी म्हणते “company X billion dollars ची झाली,” तेव्हा आता तुमच्याकडे मोजपट्टी आहे.

आता उलट्या दिशेला, छोट्या वेळाची मोजणी, कारण machine तिथे जिंकते:

1 second	तुमची एक पापणी लवते
1 millisecond (1/1000 sec)	चांगल्या internet चा एक फेरा
1 microsecond (1/1 million)	machine ने memory तून एक गोष्ट उचलणं
1 nanosecond (1/1 billion)	CPU चं एक पाऊल

एक जाड ओळ काढा: **CPU च्या एका पावलात (nanosecond) आणि internet च्या एका फेऱ्यात (millisecond) 1 million पटीचा फरक आहे.** म्हणजे एक message शहरापलीकडे जाऊन परत येईपर्यंत CPU 1 million पावलं चालून झालेला असतो. ढाब्याची उपमा: cook च्या हातात मीठ शेजारीच आहे (nanosecond), पण कांदे संपले तर मंडईची फेरी (millisecond) म्हणजे अर्धा दिवस. हुशार cook काय करतो? मंडईला रोज शंभर फेऱ्या मारत नाही; सकाळी एकदाच सगळं आणून ठेवतो. Machine चं

पूर्ण design याच शहाणपणावर चालतं: **बाहेर जाऊ नका, जोपर्यंत गरज नाही.** ही एक ओळ Part 3, 4, 5 मध्ये पुन्हा पुन्हा कामी येईल. (या फेऱ्याच्या वेळेचं नाव **latency**; हा शब्द पुढे पूर्ण chapter आणेल.)

आणि हा वेळ थेट पैसा आहे: **Google** ने मोजलं की search 400 milliseconds ने हळू झाली तर लोक कमी search करतात. अर्ध्या पापणीपेक्षा कमी वेळ, आणि करोडोंचा फरक. **Amazon** ची मोजणी: प्रत्येक 100 milliseconds ची उशीर = ~1% कमी विक्री. म्हणून वेग ही engineering ची हौस नाही, थेट पैसा आहे. जी company milliseconds मोजू शकते, ती ते रुपयांत बदलू शकते.

इथे लोक काय चुकीचं समजतात

”Computer वेगवान आहे, बस.” नाही: computer **कुठे** वेगवान आहे, हे जाणणं हीच खरी विद्या. आत nanosecond, बाहेर millisecond, 1 million पटीचा फरक. जो हे जाणत नाही, तो असं product बनवतो जे प्रत्येक पावलावर internet ला विचारतं, आणि मग चकित होतो की सगळं हळू का आहे. वेगवान machine वरही हळू design शक्य आहे, आणि सर्रास आहे.

MAP वर

या chapter मधून कोण कमावतं: जो scale वाचू शकतो. Investor साठी “1 million users” आणि “10 million users” मध्ये दहापट फरक आहे; जो founder दोन्हींना “खूप users” म्हणतो, त्याच्यापासून पैसा दूर राहतो. आणि जो engineer millisecond वाचवू शकतो, तो Google/Amazon सारख्या business मध्ये थेट revenue चा काटा फिरवतो; म्हणून अशा skill चा पगार कोटी पार जातो. आकडे जाणवणं ही Level 2, 3, 4 तिन्हींची सामायिक भाषा आहे.

स्वतः बघा (5 मिनिटं)

Phone वर stopwatch लावा आणि 100 पर्यंत मोठ्याने मोजा (~100 seconds). आता हिशोब: 1 million पर्यंत असं मोजायला किती दिवस? (~11 दिवस, न झोपता.) 1 billion पर्यंत? (~31 वर्ष.) एकदा स्वतः हिशोब केलात की billion हा शब्द कायमचा जड होईल, जसा त्याने असायला हवं.

विचार करा

1. (derivation) तुमच्या phone ची chip 15 billion transistors ची आहे. प्रत्येक transistor तांदळाचा एक दाणा असता, तर किती तांदूळ झाला असता? (एका किलोत ~50,000 दाणे.) आणि यातून समजून घ्या: chip बनवणारी factory जगात तीन-चारच का आहेत?

उत्तर: $15 \text{ billion} / 50,000 = 3$ लाख किलो, म्हणजे 300 tonnes तांदूळ, पंचवीस trucks भरून. एवढ्या गोष्टी नखाएवढ्या जागेवर, एकही चुकीची न लागता, छापायच्या आहेत. म्हणून ही जगातली सगळ्यात अवघड manufacturing आहे: एक factory उभारायचा खर्च ~20 billion dollars (1.7 लाख कोटी रुपये), आणि हुनर फक्त TSMC (Taiwan), Samsung (Korea), Intel (US) कडे. Scarcity इतकी उंच की देश यासाठी युद्धाची तयारी करतात. आता समजेल “chip war” बातम्यांमध्ये का असतं.

Chapter 2.4 [DEPTH]: एक instruction चालते कशी

(DEPTH chapter. वगळू शकता, पण हा छोटा आहे आणि यानंतर “processor” हा शब्द कायमचा उघडतो.)

Chapter 1.6 मधून: recipe numbers बनून memory त ठेवली आहे. Machine ती चालवते कशी? पूर्ण रहस्य एका मोजणाऱ्या खणात आहे.

CPU च्या आत एक छोटा खण आहे ज्यात फक्त एक number असतो: **”आता मी recipe च्या पाऊल क्रमांक ___ वर आहे.”** या खणाचं नाव **program counter**. ढाब्याचा cook आठवा, ज्याचं बोट recipe च्या कागदावर असतं: बोट ज्या ओळीवर, तेच काम चालू आणि CPU रात्रंदिवस फक्त हे तीन काम गोल-गोल करतो:

1. उचला (fetch): खणात जो number आहे, memory च्या त्या जागेवरून पाऊल उचला. समजा तिथे लिहिलंय: “3” (म्हणजे: जोडा).

2. समजा (decode): Number 3 चा अर्थ काय? Chapter 1.5 वाली gates ची साखळी number वाचून बरोबर भाग निवडते: जोडणारा डबा चालू.

3. करा (execute): जोडा. उत्तर एका खणात ठेवा. आणि मोजणाऱ्या खणात 1 वाढवा, म्हणजे पुढचा फेरा पुढचं पाऊल उचलेल.

बस. या तीन पावलांचं चक्र (**fetch-decode-execute cycle**) म्हणजेच “computer चालू आहे” चा पूर्ण अर्थ. तुमच्या phone मध्ये हे चक्र एका second ला ~3 billion वेळा फिरतं. “3 GHz processor” चा अर्थ हाच: GHz = billion फेरे प्रति second.

एक प्रश्न स्वतःला विचारा: “जर असं तर तिकडे जा” वालं पाऊल कसं चालेल? सोपं: ते पाऊल फक्त मोजणाऱ्या खणात **नवा number** लिहितं. आता पुढचा fetch तिथूनच उचलेल. Recipe ने उडी मारली. Loop पण हेच आहे: मागच्या number वर उडी, पुन्हा पुन्हा. Cook चं बोट कागदावर मागे सरकलं, बस. Programming चं पूर्ण नियंत्रण याच एका खणातून होतं.

आता रोजची एक गोष्ट या नजरेने बघा: phone कधीकधी “hang” होतो. आत काय झालं: program counter अशा जागी अडकला जिथे recipe गोल-गोल फिरते आहे आणि बाहेर पडायची अट कधीच खरी होत नाही (infinite loop), किंवा अशा number वर गेला जिथे recipe नव्हतीच. Machine बिघडली नाही; recipe ने तिला खड्ड्यात फिरवलं. Restart काय करतो? Program counter सकट सगळं सुरुवातीच्या जागेवर. म्हणून restart एवढे आजार बरे करतो.

इथे लोक काय चुकीचं समजतात

”जास्त GHz = वेगवान computer,” बस एवढंच. अर्धवट आहे. GHz फक्त चक्राचा वेग; काम किती झालं हे यावरही आहे की प्रत्येक फेऱ्यात किती होतं, पावलं किती हुशार आहेत, आणि memory किती वेगाने पावलं पुरवते आहे (पुढचा chapter). दोन processors एकाच GHz वर दुप्पट वेगळं काम करू शकतात. Truck आणि गाडीचा rpm सारखा असू शकतो; ओझं किती नेलं हे rpm सांगत नाही. दुकानदार तुम्हाला एक आकडा विकतोय; machine अनेक आकड्यांचा खेळ आहे.

MAP वर

रुपयाचा रस्ता: Apple आपल्या chips स्वतः design करते (M-series), म्हणजे प्रत्येक फेऱ्यात जास्त काम व्हावं. चांगलं चक्र = कमी battery त जास्त काम = महाग machine विकली जाते. जे चक्र तुम्ही आत्ता 10 मिनिटांत समजलात, त्याच्या बारकाव्यावर जगातल्या सगळ्यात मौल्यवान company चं margin टिकलेलं आहे. Level 3 च्या आतही मजले आहेत, आणि खालचा जेवढा चांगला, वरचं सगळं तेवढं चांगलं.

स्वतः बघा (5 मिनिटं)

ही recipe कागदावर चालवा; तुम्ही स्वतः CPU व्हा. खण K मध्ये 5 आहे. Recipe: पाऊल 1: K मध्ये 3 जोडा. पाऊल 2: जर $K < 15$, पाऊल 1 वर जा. पाऊल 3: थांबा. बोटाने program counter बना आणि चालवा. किती फेरे लागले? (K: 5, 8, 11, 14, 17 -> 4 बेरजा, मग अट खोटी, थांबलात.) तुम्ही आत्ता fetch-decode-execute आणि loop दोन्ही चालवले.

विचार करा

1. (derivation) Program counter मध्ये चुकीचा number आला, अशा जागेचा जिथे recipe नाही, कचरा numbers आहेत. CPU काय करेल? त्याला कसं कळेल की हे “recipe नाही”?

उत्तर: कळणारच नाही; हीच भीतीची गोष्ट आहे. CPU साठी प्रत्येक number पावलासारखाच दिसतो; तो कचऱ्यालाही decode करून चालवायचा प्रयत्न करेल. कधी निरुपयोगी काम होईल, कधी crash. म्हणून भोवतीची व्यवस्था (operating system, Part 2) भिंती बांधते: कुठली recipe कुठे चालू शकते, कुठली memory शिवू शकते. आणि hackers चा अर्धा खेळ हाच आहे: कसंही करून आपले numbers program counter पर्यंत पोहोचवणं. एक छोटा खण, पूर्ण security युद्धाचं मैदान.

Chapter 2.5 [SPINE]: लक्षात ठेवणं आणि विसरणं

एका कोड्याने सुरुवात. Phone वर game खेळत होतात, battery अचानक संपली. Charge करून चालू केला: game जिथे होता तिथे नाही, परत उघडावा लागला; पण तुमचे photos सगळे सुखरूप. एकाच machine मध्ये काही गोष्टी वीज गेल्यावर पुसल्या गेल्या, काही नाही. का?

कारण machine मध्ये स्मरणशक्ती **एक** नाही, **दोन** आहेत, आणि दोघींचा सौदा वेगळा:

वेगवान, विसराळू. Switches पासून बनलेली (Chapter 1.4 वाली). वीज आहे तोवर स्थिती आहे; वीज गेली, सगळं 0. पण वेग? CPU च्या पावलांसोबत धावू शकते. इथे ते राहतं जे **आत्ता** चालू आहे: उघडलेलं app, game ची आत्ताची स्थिती. हिचं नाव **RAM** (नुसतं memory पण म्हणतात).

हळू, पक्की. अशी जागा जिथे bits विजेशिवायही टिकतात. वीज जा किंवा या, data तिथेच. पण हळू: वेगवानपेक्षा शंभर-हजारपट. इथे photos, apps, files राहतात. हिचं नाव **storage** (disk, SSD).

ढाब्याची उपमा सगळं सांगते: **मेज आणि गोदाम.** Cook च्या मेजावर आत्ताच्या order चं सामान असतं: चिरलेला कांदा, हाताशी मसाले. मेज छोटं आणि महाग (जागा मर्यादित), पण हात लगेच पोहोचतो. गोदाम मोठं आणि स्वस्त, पण फेरी लागते. रात्री ढाबा बंद झाला की मेज पुसलं जातं; गोदाम तसंच राहतं.

आता विचारा: दोन्हींचे फायदे असलेली एक गोष्ट का नाही बनवत? वेगवान पण, पक्की पण, स्वस्त पण? Engineers 70 वर्षं प्रयत्न करतायत. Physics चा सौदा आहे: **वेगवान, स्वस्त, पक्की: तिन्हीपैकी दोनच निवडा.** म्हणून machine दोन्ही ठेवते आणि मध्ये data ची ने-आण चालू राहते:

App “उघडणं” = गोदामातून मेजावर recipe copy करणं. म्हणून उघडायला वेळ लागतो. File “save करणं” = मेजावरून गोदामात लिहिणं. म्हणून save चा अर्थ: “आता वीज गेली तरी वाचेल.” Phone “hang” = मेज भरलं, आता machine प्रत्येक पावलाला गोदामाच्या फेऱ्या मारते आहे.

खोक्यावरची ओळ आता पूर्ण उघडते: “8 GB RAM, 128 GB storage” म्हणजे 8 GB चं कामाचं मेज, 128 GB चं पक्कं गोदाम. आणि दुकानदाराची चाल पण दिसते: स्वस्त phone वर तो storage चा मोठा आकडा दाखवतो (128 GB!) आणि RAM लपवतो (4 GB). Phone अडखळतो तो RAM कमी म्हणून, storage मुळे नाही. आता तुम्ही खोका वाचून खरा प्रश्न विचारू शकता: मेज केवढं आहे? हे छोटं ज्ञान हजारो रुपयांच्या चुकीच्या खरेदीपासून वाचवतं.

इथे लोक काय चुकीचं समजतात

"Memory full झाली, जुने photos delete करा." दोन वेगळ्या गोष्टी मिसळल्या: photos गोदामात (storage) आहेत, hang होणं मेजाचं (RAM) आहे. Photos हटवल्याने hang बरा होत नाही (गोदाम अगदी ठासून भरलं असेल तरच थोडा फरक). आणि उलटंही: "8 GB RAM आहे तर 8 GB ची file आरामात उघडेल" पण सरळ नाही; मेजावर आधीच इतर गोष्टी मांडलेल्या असतात. दोन नावं, दोन कामं; आता कायम वेगळी ठेवा.

MAP वर

Company case: memory chips चा बाजार (Samsung, SK Hynix, Micron) ~150 billion dollars चा आहे, आणि AI ने त्याला आग लावली आहे, कारण AI recipes चं मेज (RAM) राक्षसी आकाराचं लागतं. 2024-25 मध्ये AI च्या memory chips ची (HBM नावाच्या) इतकी टंचाई झाली की भाव आभाळाला गेले. Chapter 0.2 चं सूत्र: scarcity चा काटा. जेव्हा नवी technology येते, तेव्हा तिच्या खालच्या कुठल्यातरी भागाची टंचाई कोणाचं तरी सोनं बनते. हा pattern ओळखणं म्हणजेच संधी ओळखणं.

स्वतः बघा (5 मिनिटं)

एखादं जड app उघडा (game किंवा editor): किती seconds लागले? हा गोदाम-ते-मेज copy चा वेळ होता. आता ते बंद करून **लगेच** परत उघडा: यावेळेस पटकन उघडलं ना? कारण system ने ते मेजाजवळच ठेवलं होतं. तुम्ही आता दोन्ही स्मरणशक्तींमधला फरक स्वतःच्या घड्याळाने मोजला.

विचार करा

1. (derivation) या chapter मधून एका गोष्टीचं उत्तर काढा: "Save" button जगातून हळूहळू गायब का होतोय? (Google Docs मध्ये save button नाहीच.)

उत्तर: आधी save यासाठी होतं की मेजावरून गोदामात लिहिणं महाग काम होतं; मग machine तुम्हाला विचारायची कधी लिहू. आता storage वेगवान आणि स्वस्त झाला, मग software दर काही seconds नी चुपचाप स्वतःच लिहितं (auto-save), आणि internet वरही copy पाठवतं. जेव्हा एखाद्या कामाचा खर्च कोसळतो, तेव्हा त्याच्याशी जोडलेला button पण मरतो. Design ला technology ची economics चालवते; product बनवताना ही गोष्ट नेहमी सोबत ठेवा.

PART 1 चा शेवट: नकाशा आणि पुढचं पाऊल

जे तुमच्याकडे आता आहे

पैसा	विश्वासाची व्यवस्था; फक्त Level 1 वर घुसतो
कर्माईचं सूत्र	size x scarcity x leverage (तास नाहीत)
चार LEVELS	गरज -> business -> technology -> AI; पैसा खालून वर वाहतो
ABSTRACTION	प्रत्येक मजला खालचा लपवतो, आणि लपलेला हिस्साच महाग असतो
MACHINE	billions switches (transistors), design केलेल्या साखळ्यांमध्ये
BIT / BYTE	switch ची स्थिती / 8 switches चा गट
GATES	AND, OR, NOT: यांतूनच बेरीज, तुलना, "जर"
COMPUTER	numbers वर आधीच लिहिलेली recipe, billion पावलं प्रति second, न थकता
UNIVERSAL MACHINE	recipe पण number आहे, म्हणून एक machine सगळं काही: हाच software चा जन्म
सगळं NUMBER	text/photo/आवाज/video: कापा, मोजा, अर्थ आधीच ठरवा (encoding)
माप	KB->MB->GB->TB, प्रत्येक पायरी हजारपट; text : photo : video = 1 : हजार : लाख
आत VS बाहेर	CPU चं पाऊल nanosecond, internet चा फेरा millisecond: 1 million पटीचा फरक
दोन स्मरणशक्ती	RAM (मेज: वेगवान, विसराळू) vs storage (गोदाम: हळू, पक्कं)

एक परीक्षा, स्वतःसाठी

पुस्तक न उघडता, कोणाला तरी (किंवा स्वतःला, बोलून) ही 4 उत्तरं द्या:

1. पैसा फक्त Level 1 वर का घुसतो?
2. Surgeon पेक्षा app वाला जास्त का कमावतो?
3. एकच phone camera पण, bank पण, game पण कसा बनतो?
4. "8 GB RAM, 128 GB storage" चा अर्थ काय?

चारही आली, तर Part 2 साठी तयार आहात. जे अडलं, त्याचा chapter number वरच्या नकाशात मिळेल; तिथे परत जा.

Part 2 मध्ये काय आहे

Machine recipe चालवते, हे कळलं. आता पुढचा प्रश्न: recipe लिहिली कशी जाते? माणसाच्या भाषेपासून machine च्या numbers पर्यंत भाषांतर कोण करतं? Languages इतक्या साऱ्या का आहेत? तो manager कोण जो सगळ्या recipes ना एका machine वर भांडण्यापासून रोखतो (operating system)? आणि software मध्ये bugs असतातच का? Part 2: MACHINE शी बोलणं.

PART 1 चा MINI-GLOSSARY

फक्त ते शब्द जे या part मध्ये आले. प्रत्येकापुढे त्याचा chapter, म्हणजे परत जाणं सोपं.

abstraction	आतली गुंतागुंत झाकून सोपं handle देणं (0.4)
AND / OR / NOT	तीन मूळ gates: दोन्ही/कोणीही/उलट (1.5)
ASCII / Unicode	अक्षरांच्या numbers चा करार, जुना / आजचा (2.2)
binary	दुप्पट-वाली मोजणी, फक्त 0 आणि 1 (1.4)
bit	एका switch ची स्थिती: 0 किंवा 1 (1.4)
byte	8 bits; साधारण एक अक्षर (1.4)
circuit	विजेचा बंद रस्ता (1.1)
CPU / processor	recipe चालवणारा मुख्य भाग (1.3)
double coincidence of wants	दोघांना एकाच वेळेस एकमेकांची गोष्ट हवी (0.1)
encoding	numbers चा अर्थ ठरवणारं table (2.2)
fetch-decode-execute	CPU चं तीन-पावलांचं चक्र: उचला, समजा, करा (2.4)
frame	video चा एक photo; ~30 प्रति second (2.2)
GHz / clock speed	चक्राचा वेग: billion फेरे प्रति second (2.4)
hardware	हाताला लागणारी machine (1.3)
KB / MB / GB / TB	size ची शिडी, प्रत्येक पायरी हजारपट (2.1)
latency	काम सुरू होऊन उत्तर येईपर्यंतचा वेळ (2.3)
ledger	देवाण-घेवाणीची वही; पैसा चालती वही आहे (0.1)
leverage	काम एकदा, फायदा पुन्हा पुन्हा (0.2)
logic gates	switches पासून बनलेले ठरवणारे भाग (1.5)
market size	problem किती लोकांची, किती खोल (0.2)
pixel	photo चा एक ठिपका; तीन numbers चा रंग (2.2)
program	machine साठी लिहिलेली recipe (1.3)
program counter	"आत्ता पाऊल क्रमांक __ वर" वाला खण (2.4)
RAM	वेगवान, विसराळू स्मरणशक्ती; कामाचं मेज (2.5)
sample	आवाजाच्या एका क्षणाचं माप; ~44,000/second (2.2)
scarcity	हे किती कमी लोकांना जमतं (0.2)
software	सगळ्या recipes, मिळून (1.3)
stack	मजल्यांचा मनोरा: कोण कोणावर उभा (0.3)
storage	हळू, पक्की स्मरणशक्ती; गोदाम (2.5)
stored-program computer	recipe ला data च्या जागी ठेवणारं design (1.6)
transistor	विजेने दाबला जाणारा switch (1.1)
universal machine	एक machine जी कुठलीही recipe चालवते (1.6)